OptiMUS-0.3: Using large language models to model and solve optimization problems at scale

Ali AhmadiTeshnizi[a] , Wenzhi Gao[b], Herman Brunborg[b] Shayan Talaei[a], Connor Lawless[a], Madeleine Udell[a,b]

[a] School of Management Science and Engineering, Stanford University, Stanford, California 94305

[b] Institute for Computational and Mathematical Engineering, Stanford University, Stanford, California 94305

arXiv:2407.19633v4 [cs.AI] 26 Jun 2026

Abstract. Optimization problems are pervasive in sectors from manufacturing and distribution to healthcare. However, most such problems are still solved heuristically by hand rather than optimally by state-of-the-art solvers because the expertise required to formulate and solve these problems limits the widespread adoption of optimization tools and techniques. We introduce a Large Language Model (LLM)-based system designed to formulate and solve (mixed integer) linear programming problems from their natural language descriptions. Our system can develop mathematical models, write and debug solver code, evaluate the generated solutions, and improve efficiency and correctness of its model and code based on these evaluations. OptiMUS is designed as a productivity tool for optimization practitioners who understand the problem domain and can describe it precisely, but seek to accelerate the modeling and implementation workflow. OptiMUS-0.3 utilizes a modular structure to process problems, allowing it to handle problems with long descriptions and complex data without long prompts. Experiments demonstrate that OptiMUS-0.3 outperforms direct-prompting baselines by over 43% on easy and 18% on hard instances. It remains competitive with fine-tuned specialist models on benchmark problems, and outperforms them on real-world case studies (28.6% vs. 0%) where fine-tuned models fail to generalize. Ablation studies show that modular architecture with error correction is central to these gains. A key finding is that system architecture is a stronger driver of performance than model capability. Structured decomposition with targeted error correction enables weaker models to match stronger models under naive prompting, and remains competitive with fine-tuned specialist models without retraining costs.

Key words: Large Language Models, Optimization Modeling, AI Agents, Mixed Integer Linear Programming, Optimization Tools, LLM System Design

1. Introduction

Optimization problems are ubiquitous in domains from operations and economics to engineering and computer science. While major advances in optimization algorithms over the last several decades have led to reliable and efficient optimization methods for a wide variety of structured optimization problems, including linear programming (LP) and mixed-integer linear programming (MILP), optimization modeling — transforming a domain problem into a mathematical optimization problem — still requires expert knowledge. According to a recent survey, 81% of Gurobi's commercial solver users have advanced degrees, with 49% of them holding a degree in operations research (Gurobi Optimization 2023). This expertise gap prevents many organizations from using optimization, even when it could significantly improve their operations. Examples include inventory management in supermarkets, patient operations in hospitals, transportation policies in small municipalities, energy management in local solar farms, and operations in small businesses or NGOs

(Saghafian et al. 2015, Aastrup and Kotzab 2010, Yao et al. 2020, Shakoor et al. 2016). Automating optimization modeling would allow optimization practitioners to tackle more problems and develop models faster, amplifying the impact of existing expertise.

In this paper, we develop an automated system to accelerate the workflow of existing optimization practitioners. We specifically focus on helping optimization users develop, tune, and implement (mixed-integer) linear optimization problems, a flexible and broad class of optimization problems that have been applied successfully in a wide range of real-world use cases such as scheduling and production planning. Our work aims to improve the productivity of optimization practitioners—for example, an MBA analyst in industry who understands the problem domain but wants to model and implement faster—akin to the productivity gains realized by software engineers with systems such as GitHub CoPilot. This work is a step towards systems that could ultimately broaden access to optimization tools for users with less formal training, but that goal is beyond the scope of the current paper. Given that our focus is on existing optimization practitioners, we assume for the remainder of this paper that the user has already identified a problem for which MILP is an effective tool and can describe the problem in a level of detail such that modeling is possible. Aiding users to find the correct modeling paradigm (e.g., constraint programming, conic optimization) and supporting them in concretizing vague problems remains an important open problem for future research.

Large language models (LLMs) offer a promising way to automate optimization modeling, and increase the reach of the powerful algorithms developed by the operations research community. However, using LLMs for optimization modeling presents unique challenges that arise from the scale and complexity of real-world optimization problems. Scalability in optimization modeling arises along several distinct dimensions: (i) *description length*, the length of the natural-language problem description; (ii) *problem complexity*, the number and type of decision variables, constraints, and logical or combinatorial structures (e.g., binary variables, special ordered sets, logical constraints); and (iii) *data scale*, the size of the structured input data (e.g., number of customers, products, or time periods). In this work, we provide empirical evidence of scalability with respect to description length and problem complexity, and present initial evidence on data scale. Beyond these scalability challenges, LLMs still suffer from major flaws that prevent deployment in important applications. We highlight four challenges:

- **Long problem descriptions.** Realistic optimization problems can be exceedingly long and complex: for example, the documentation for the energy system problem described in Holzer et al. (2023) is 60 pages long. Unfortunately, LLMs have a limited context size, and even long-context models perform less well as the input context grows (Liu et al. 2023, Levy et al. 2024). Consequently, LLMs tend to make more mistakes as the length of the problem description increases and perform poorly on complex problems.
- **Large problem data.** Optimization models frequently depend on structured input data such as customer attributes, network structures, or time-indexed parameters. As the size of these datasets grows, modeling systems must correctly incorporate this data into the formulation without exceeding prompt limits or introducing modeling errors.

- **Hallucination.** LLMs are known to *hallucinate*: they may produce answers that sound reasonable, but are incorrect. In the context of optimization, the generated solver code may hallucinate constraints that incorrectly model the problem, or it may hallucinate API calls that do not exist, resulting in code that cannot run. It is especially challenging to verify whether the solution is correct, supposing the code runs without error. For instance, if the solver claims the solution is unbounded, perhaps a constraint has been accidentally omitted from the formulation.
- **Bad models.** The solve time for an optimization problem can depend on the particular modeling formulation chosen, and on how the structure of the problem is communicated to the solver. Optimization experts spend much of their effort modeling the problem to enhance the efficiency of the solution method. A challenge for LLMs is to produce not just an accurate model, but good code that solves the problem quickly.

This paper studies the feasibility of using LLMs to expand the reach of operations research techniques and analytic methods for decision making by studying the performance of OptiMUS, an automated system we have developed that uses LLMs combined with traditional solvers to model and solve (mixed integer) linear programs (MILPs). Our studies of OptiMUS demonstrate both the promise and pitfalls of the LLM approach to quantitative decision making, and help clarify the effectiveness of LLMs for optimization modeling as of 2026. More broadly, through ablation studies, this paper shows how structured reasoning pipelines provide a clear pathway to address current limitations and enhance the reliability of LLM-based systems in optimization. Through systematic ablation studies, we find that the primary bottleneck in LLM-assisted optimization modeling is system architecture, not LLM capability. Modular decomposition with targeted error correction enables weaker models to match stronger models using naive prompting, and matches fine-tuned specialist models without retraining costs.

We organize our evaluation of scalability around the three dimensions introduced above.

- **Description length.** We demonstrate scalability with respect to description length through the NLP4LP benchmark (Table 2). Problems in the hard subset have descriptions averaging 912 characters (nearly twice the length of the easy subset), and the case study instances average 2,578 characters, an order of magnitude longer than textbook instances. Performance on these subsets provides direct evidence of the system's ability to handle increasingly long problem descriptions.
- **Problem complexity.** We evaluate scalability with respect to problem complexity through the seven case-study instances (Section 3), which are drawn from published real-world applications and involve rich combinatorial structure: multiple classes of integer and binary variables, logical constraints, and domain-specific modeling patterns that differ substantially from textbook problems.
- **Data scale.** OptiMUS handles large input data by separating problem data from the natural-language description: data is stored externally and referenced symbolically in the model, so prompt length does not grow with data size. We validate this empirically in Section 5.4: comparing OptiMUS against a naive

prompting baseline (full data provided in prompt) across input sizes from 3 KB to 935 KB, OptiMUS maintains 100% success rate throughout while naive prompting degrades and fails entirely once data exceeds GPT-4o's context window.

We do not make claims about scalability along other dimensions, such as solver-level scalability (handling models with thousands of variables or constraints). These are important directions for future work but are outside the scope of the current empirical evaluation.

Our contributions in this paper are as follows:

- We curate NLP4LP, a comprehensive open-source dataset of 361 optimization problems organized into three subsets: an *easy* subset of 289 LP problems with short descriptions and scalar parameters; a *hard* subset of 65 LP/MILP problems with longer descriptions and multi-dimensional parameters; and a *case study* subset of 7 real-world MILP problems drawn from the INFORMS Journal on Applied Analytics, featuring complex combinatorial structure and large-scale input data. Table 2 compares NLP4LP to existing datasets and Section 3 describes NLP4LP in detail. Our dataset also includes problems that are less likely to have been used to train existing LLMs, reducing the risk of data leakage compared to previous optimization modeling datasets.
- We develop a modular, LLM-based agent to model and solve optimization problems, which we call OptiMUS (or, occasionally, OptiMUS-0.3 to distinguish it from prior versions). OptiMUS-0.3 employs a connection graph that allows it to process each constraint and objective independently, which allows it to process problems with long natural-language descriptions and large input datasets without requiring excessively long prompts.
- We develop several modules designed to improve the performance of OptiMUS and study their impact, including:
  1. *Self-reflective error correction.* We ask OptiMUS to evaluate and correct its output, and assess its own confidence in its output, allowing it to fall back to a more powerful LLM or to user feedback when OptiMUS is unsure its answer is correct.
  2. *Advanced optimization modeling techniques.* We teach OptiMUS advanced optimization modeling techniques by prompting it to identify important structures in the problem, including structure of problems and solver features such as special ordered sets, and to use those structures to model and solve the problem more efficiently.
- We perform ablation studies of the components of our framework to demonstrate which elements improve performance and which can actually degrade performance. We find OptiMUS-0.3 beats GPT-4o alone by over 43% on easy instances and over 18% on hard instances. On seven real-world case-study instances, OptiMUS-0.3 (GPT-4o) solves 28.6% of instances, compared to 14.3% for direct prompting with GPT-4o and 0% for fine-tuned specialist models.

- We develop a publicly available web application that allows practitioners to try OptiMUS. This webapp enables human-in-the-loop automated optimization modeling: users can edit LLM outputs and OptiMUS can flag low-confidence LLM outputs to solicit corrections from the user.

An initial version of this work was published in a conference proceeding (AhmadiTeshnizi et al. 2024), that introduced a smaller version of the NLP4LP dataset, and a less robust LLM agent-based system for modeling optimization problems that we will refer to as OptiMUS-0.2. The new enlarged version of the NLP4LP dataset which accompanies this paper has been open-sourced and includes a modular design that enables future researchers to improve on different sub-problems within our framework such as parameter extraction, modeling, or coding. Compared to the earlier version, the modular design of OptiMUS-0.3 substantially improves the robustness of the system for more complicated modeling problems. New modules that enable the system to leverage advanced solver functionality, such as special-order-set constraints, can dramatically speed up the solution time for the associated optimization problems. Additional LLM techniques in OptiMUS-0.3 such as self-reflective error correction improve performance. Finally, this paper is the first to present the OptiMUS webapp.

2. Related Work

Formulating an optimization model is often a challenging task even for experts in optimization. Different formulations can lead to significantly different solving times and enable the use of different solvers or solution techniques (Boyd and Vandenberghe 2004). One important skill for an optimization expert is to identify assumptions or relaxations that allow for casting the problem as a well-studied problem type, such as MILP, which enables the use of well-developed optimization solvers. Crafting such an efficient formulation often requires specialized knowledge (Zohrizadeh et al. 2020, Low 2013, Roubíček 2020, Luo et al. 2010, Krarup and Pruzan 1983).

Given the formulation, an optimization expert must choose a solver. Each solver has a distinct interface and capabilities, with associated benefits and downsides (Achterberg 2019, Diamond and Boyd 2016, CPLEX User's Manual 1987). However, the user manuals for these solvers are often hundreds of pages, making them difficult to understand and use.

Progress in LLMs. Recent progress in Natural Language Processing (NLP) has led to the development of LLMs useful for tasks such as answering questions, summarizing text, translating languages, and coding (OpenAI 2023, Touvron et al. 2023, Chowdhery et al. 2022, Wei et al. 2023, Gao et al. 2023, Borgeaud et al. 2022). Connections to other software tools extend the reach and accuracy of LLMs, as demonstrated by plug-ins for code writing and execution (Paranjape et al. 2023, Wei et al. 2023). Yang et al. (2023) use LLMs to directly generate solutions to optimization problems without calling traditional solvers through prompt optimization to improve performance. The approach is limited to small problems since the performance of LLMs degrades as the input context grows, even for explicitly long-context models (Liu et al. 2023).

Chatbots for Optimization. In a recent paper, Chen et al. (2023) developed a chatbot to help users detect and fix infeasible optimization problems expressed in `Pyomo` code and serves as an AI assistant rather than as a solver. Li et al. (2023) designed a chatbot to answer natural-language queries about a supply chain optimization model. Alibaba Cloud (2022) also developed a chatbot to facilitate optimization modeling, but there is no public paper or documentation available on it. Lawless et al. (2024) explored the use of LLMs to allow users to customize a simple constraint programming model in the context of meeting scheduling, but does not model more general MILP optimization problems.

Benchmark-driven Optimization Modeling. More closely related to our approach, Ramamonjison et al (2023) introduced a dataset of 1101 natural language representations of LP problems. They proposed a two-stage mapping from the natural-language representation to the problem formulation using an intermediate representation. Ramamonjison et al (2022) designed a system to simplify and improve the modeling experience for operations research, but did not offer an end-to-end solution. Xiao et al. (2023) presented a multi-agent cooperative framework called Chain of Experts (CoE) to automatically model and program complex operation research (OR) problems, and evaluated it on NL4Opt and another more complex dataset, ComplexOR, introduced in that paper. Astorga et al. (2024) present a Monte-Carlo Tree Search (MCTS) based approach that decomposes the modeling problem into stages (e.g., modeling variables, then objectives, then constraints, etc.) and uses MCTS to explore the space of plausible models. As mentioned in the paper, this approach is not directly comparable to OptiMUS as it outputs a set of functionally distinct models as opposed to a single formulation and answer. Concurrent with the preparation of this paper, Tang et al. (2024) introduced a new approach to OR modeling with LLMs that uses fine-tuning on a semi-synthetic dataset rather than prompt optimization or agentic models, which they call ORLM (Operations Research Language Model). Yang et al. (2024) also explore a synthetic data generation and fine-tuning approach for a broader class of optimization problems. These fine-tuning approaches are complementary to our work, as any progress towards more capable LLMs for optimization modeling can be combined with our modular prompting framework.

Traditional MILP solvers generally benchmark against the `MIPLIB` benchmark (Gleixner et al. 2021), which offers a diverse collection of MILP problems in standard form. Unfortunately, most of these problems are not associated with a natural-language description, and so cannot be used to study optimization modeling as we do in this paper.

3. Dataset

As part of this paper, we introduce a comprehensive open-source dataset of 361 optimization problems we call NLP4LP. The dataset builds upon earlier datasets for automated modeling, notably the NLP4OPT dataset Ramamonjison et al (2023), but includes a much richer set of information about the instances including an optimal solution and associated code. Our goal in creating NLP4LP is to provide the community with

examples they can use to design optimization modeling tools using natural language systems and to assess their quality. The problems in NLP4LP are partitioned into three subsets: 1) an easy dataset that contains only LP problems with short descriptions and scalar parameters, 2) a hard dataset that contains both LP and MILP problems with longer descriptions and multi-dimensional parameters, and 3) a case study dataset drawn from real-world applications published in the INFORMS Journal on Applied Analytics (Table 2). Unlike other modeling datasets, each instance in the NLP4LP dataset is accompanied by associated code to run the instance, and ground-truth intermediary representations of the problem (i.e., extracted parameters and targets, a list of clauses of the problem represented in natural language, LaTeX, and code, and a solution and optimal value for given problem data). This additional structure enables future researchers to investigate and improve different modules of the OptiMUS framework such as parameter extraction or code implementation, and even test these improvements with the OptiMUS webapp, without implementing the full software stack from scratch. Compared to the initial version of the NLP4LP dataset presented in AhmadiTeshnizi et al. (2024), this version contains more instances from a wider breadth of domains and more complex problems from published papers, and includes additional information for each instance, such as code, as discussed earlier. The dataset is available on Hugging Face and in the supplementary material.

Given the complexity of the task, and the components of each instance (description, sample feasible data, and solution code), gathering high-quality optimization instances is expensive. Unlike traditional ML datasets, NLP4LP does not provide a training set with the same distribution as the test set. As the cost of collecting each instance is high, we prioritize creating a large test set rather than reducing the size of the test set to provide training data. Consequently, the easy and hard subsets together comprise 354 problems, partitioned into 1) a development (dev) set of 23 problems (12 easy and 11 hard) and 2) a test set of 331 problems (277 easy and 54 hard). The dev set is intended to be used to develop optimization modeling tools, while the test set should be used to evaluate such tools. In our experiments and system design, we did not use any part of the test set for development, ensuring an unbiased evaluation of the models.

The third subset consists of 7 case-study instances drawn from real-world operations research applications published in the INFORMS Journal on Applied Analytics (see Table 1). Each instance was manually formulated and solved by the authors based on the corresponding published paper, with synthetic data generated to match the scale described therein. These problems are substantially longer and more complex than the easy and hard subsets, featuring multi-indexed parameters, rich domain-specific structure, and descriptions averaging 2,578 characters.

Optimization modeling is used in many domains. To ensure that NLP4LP comprehends a wide variety of use cases, we first created a list of important types of optimization problems, such as scheduling, cutting, routing, blending, and packing, and a list of common application domains such as sports, government, retail, agriculture, and energy. We tagged each problem NLP4LP with all relevant labels, and continued to gather instances until the dataset contains at least two instances per label. Instances in our dataset are drawn from

Table 1 Real-world case studies in the NLP4LP case study subset.

ID	Domain	Description	Reference
355	E-commerce	**Amazon Locker Capacity Management**: maximize locker throughput by reserving capacity per shipping option subject to capacity and demand constraints.	Sethuraman et al. (2024)
356/357	Education	**Hybrid Class Scheduling**: two variants minimizing excess in-person attendance and maximizing student interaction, respectively, under social distancing capacity limits.	Moallemi and Patange (2024)
358	Social Policy	**Human Trafficking Interception**: evaluate transit monitoring station efficiency via Data Envelopment Analysis.	Dimas et al. (2023)
359	Urban Services	**Waste Collection Routing**: minimize total route length for a city-wide waste collection vehicle fleet.	Bertero et al. (2023)
360	Retail Logistics	**Rich Vehicle Routing**: minimize delivery cost for a DIY retailer using dedicated LSP tours and common carriers with a heterogeneous fleet.	Tuma et al. (2024)
361	E-commerce	**Order Fulfillment Consolidation**: maximize multiorder hits using an LP-based pool management policy with machine-learned consolidation probabilities.	Wang et al. (2024)

several sources: some were created by our research team, inspired by problems in textbooks, lecture notes, and research papers, and some are drawn from existing datasets on LP modeling (Bertsimas and Tsitsiklis 1997, Williams 2013, Nace 2020, Xiao et al. 2023, Ramamonjison et al 2023). Most problems have around four labels, and some have as few as two or as many as eight. For more information on problem types and domains refer to Appendix G. Real-world optimization problems are substantially longer and more complex than textbook instances. Table 2 shows that the NLP4LP Hard subset (avg. 912 characters) and Case Study subset (avg. 2,578 characters) contain problems with descriptions far longer than those in existing MILP modeling datasets, and that the case-study instances seriously challenge the modeling capabilities of current LLM systems (Section 5.4).

One major concern in evaluation of LLM-based systems is data leakage: has the LLM been trained on the data in the purported test set? This issue is serious and has been known to contaminate the results of several important studies (Oren et al. 2023). Our goal is not to claim that the dataset is completely original (as it builds upon previous datasets), but rather ensure that all instances in our dataset have not been leaked to model pre-training. In our study, we take several measures to mitigate the risk of leakage, balancing this concern against the goal of broad coverage in our test set.

Table 2 A comparison on different aspects of complexity for various datasets. The unit for description length is characters

Dataset	Description Length	Instances (#MILP)	Multi-dimensional Parameters
NL4Opt	518.0 ± 110.7	1101 (0)	×
ComplexOR	497.1 ± 247.5	37 (12)	✓
NLP4LP Easy (Ours)	507.2 ± 102.6	289 (0)	✓
NLP4LP Hard (Ours)	912.3 ± 498.2	65 (18)	✓
NLP4LP Case Study (Ours)	2578.0 ± 976.5	7 (7)	✓

- Our dataset is guarded by Captcha, and only authenticated Hugging Face users can access it after agreeing to terms and conditions. These safeguards prevent the dataset from being used to train future LLMs.
- Many problems in the dataset are inspired by textbook examples and recent publications, yet their text is entirely original. Moreover, the sources of these questions do not include solutions (either in LaTeX or code). We verified that the content of these questions does not appear anywhere on the internet using a plagiarism detection tool, which yielded a mean originality score of 84.9% and a median originality score of 100%. See Appendix G for details.
- The easy subset of NLP4LP is built directly on top of the NL4Opt dataset (Ramamonjison et al 2023): the problem descriptions and formulations are the same, but each instance has been extended with the additional annotations required for our benchmark, namely associated solution code, ground-truth optimal values and solutions, extracted parameters, and clause-level natural language representations. This enrichment enables future researchers to evaluate and improve individual components of the modeling pipeline (e.g., parameter extraction, clause modeling, code generation) independently, which NL4Opt does not support. None of the MILPs, which represent the most challenging problems in our dataset, are sourced from existing datasets.

4. Methodology

An optimization problem proposes to maximize or minimize a given function f over a set of allowable inputs $\mathcal{X}$. This paper focuses on modeling mixed-integer linear optimization problems (MILP) where f is a linear function and the set of allowable inputs $\mathcal{X}$ can be represented by a set of linear inequalities over integer and real valued variables. A given MILP comprises three key components: (1) a set of n discrete and real variables (i.e., the inputs that can be changed), (2) a set of linear inequalities and equalities that define different constraints on the input variables, and (3) a linear objective function that defines what the problem aims to minimize or maximize. A MILP can be written mathematically as

$$\underset{\{x\}}{\text{minimize}} \quad \sum_{j=1}^{n} c_j x_j$$

$$\begin{aligned} \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j \leq b_i, \quad i = 1, \dots m_1 \\ & \sum_{j=1}^{n} a_{ij} x_j = b_i, \quad i = 1, \dots m_2 \\ & x \in \mathbb{Z}^{n-r} \times \mathbb{R}^r. \end{aligned}$$

A feasible point x^* that minimizes the objective function is called a (optimal) solution and has associated optimal objective value $c^T x^*$. For brevity, we refer to the equations defining constraints and the objective as different *clauses* in the optimization model. A *parameter* of an optimization model is any numeric coefficient in the optimization model (i.e., elements of A or c). The goal of the OptiMUS system is to go from a natural language description of an optimization problem (e.g., a paragraph of text describing a business problem), which we call the *problem description*, to a decision (assignment of numerical values to each variable) that solves the problem. We assume that the problem description contains all the information needed to model the optimization problem. In practice, users may give ambiguous or incomplete descriptions of an optimization problem (see Wasserkrug et al. (2024), Lawless et al. (2024) for a discussion). Helping users to resolve these ambiguities automatically is an exciting direction for future work.

OptiMUS-0.3 decomposes these two tasks into a sequence of LLM-powered steps that incrementally constructs an optimization model and implements it. An overview of the complete workflow of OptiMUS-0.3 is outlined in Fig. 1. At a high level, OptiMUS-0.3 begins by 1) identifying parameters from the problem description and writing a natural-language description of each clause in the problem. It then 2) formulates both the clauses and decision variables of the optimization model with mathematical precision, using LaTeX code. Finally, 3) OptiMUS-0.3 uses the LaTeX model of the optimization problem and the extracted parameters to generate code snippets for each clause in Python and assembles the snippets into a single runnable code file. By running this file, OptiMUS-0.3 outputs an optimal solution and optimal objective value. Throughout this process, OptiMUS-0.3 maintains a *state* that documents what is known about the optimization model, as well as a *connection graph* that tracks which variables appear in which constraint (detailed in Section 4.1). The current state is supplied in the prompt of every LLM-powered component (detailed in Section 4.2). Each LLM-powered component of OptiMUS-0.3 also incorporates an Error Correction (EC) module (detailed in Section 4.3) that catches common errors and improves the accuracy of the modeling and implementation.

Our current implementation writes code that uses Gurobi (Achterberg 2019), a leading MILP solver, and its associated Python API gurobipy. However, we have designed OptiMUS-0.3 to be easy to extend to other modeling languages and backends. We expect it to perform equally well with other solvers (e.g., CPLEX, SCIP) that have well-documented APIs, as well as with other modeling languages such as cvxpy (Diamond and Boyd 2016).

Many of the advances in optimization solvers in the last several decades rely on the fact that solvers are faster when they can exploit particular structures within the optimization problem. One of the main tasks of an optimization expert is to identify these structures so as to choose an appropriate solver or parameter settings. However, detecting useful structures in a given optimization problem can be very challenging, since it requires both a deep understanding of the mathematical structure of the problem and the capabilities of existing solvers. In view of this challenge, OptiMUS-0.3 includes two specialized optimization modules: 1) a Structure Detection Agent, that detects variables (e.g., Indicator variables) and constraints (e.g., SOS constraints) with special structure that can be exploited by the solver, and 2) a customized optimization coding agent, that can leverage callbacks in the solver and perform basic column and constraint sifting to help improve the runtime of the code. These elements represent a first step towards automating more advanced decomposition algorithms with an automated modeling framework.

OptiMUS-0.3 can be run as an end-to-end pipeline or through an interactive web application designed to facilitate human-system collaboration. The OptiMUS web application uses the same underlying components as the end-to-end framework but provides additional opportunities for users to audit and edit the outputs of each stage of the OptiMUS pipeline. We defer a deeper discussion of the system to Section 6, and mention specific interaction mechanisms available in the user interface earlier when relevant.

The remainder of this section details the key components of the OptiMUS-0.3 framework. Section 4.1 describes the underlying state and connection graph of the system. Section 4.2 describes a sample LLM component of the system. The remaining sections describe specialized modules that improve the performance of the system, including error correction and debugging (Section 4.3), specialized modeling tools (Section 4.4), and advanced optimization coding (Section 4.5). OptiMUS-0.3, and all associated components, have been released as open-source codeand are publicly available to use through a web application, described in Section 6.

4.1. State

OptiMUS-0.3 manages and modifies the solution using states saved in JSON format. The state consists of the following components:

- **Parameters**: OptiMUS-0.3 can choose a symbol for each parameter, infer its shape, and define the parameter if it is not explicitly included in the problem statement. Numerical data from the problem statement is replaced by a symbol and stored separately for later use, ensuring that the parameter list remains concise and easy to include in future prompts.
- **Clauses**: Each clause (objective or constraint) consists of a natural language description, a LaTeX formulation, and code implementing the clause.
- **Variables**: Like parameters, each variable has a symbol, a shape, and a definition. Unlike parameters, each variable also has a type, which can be *Continuous*, *Integer*, or *Binary*.

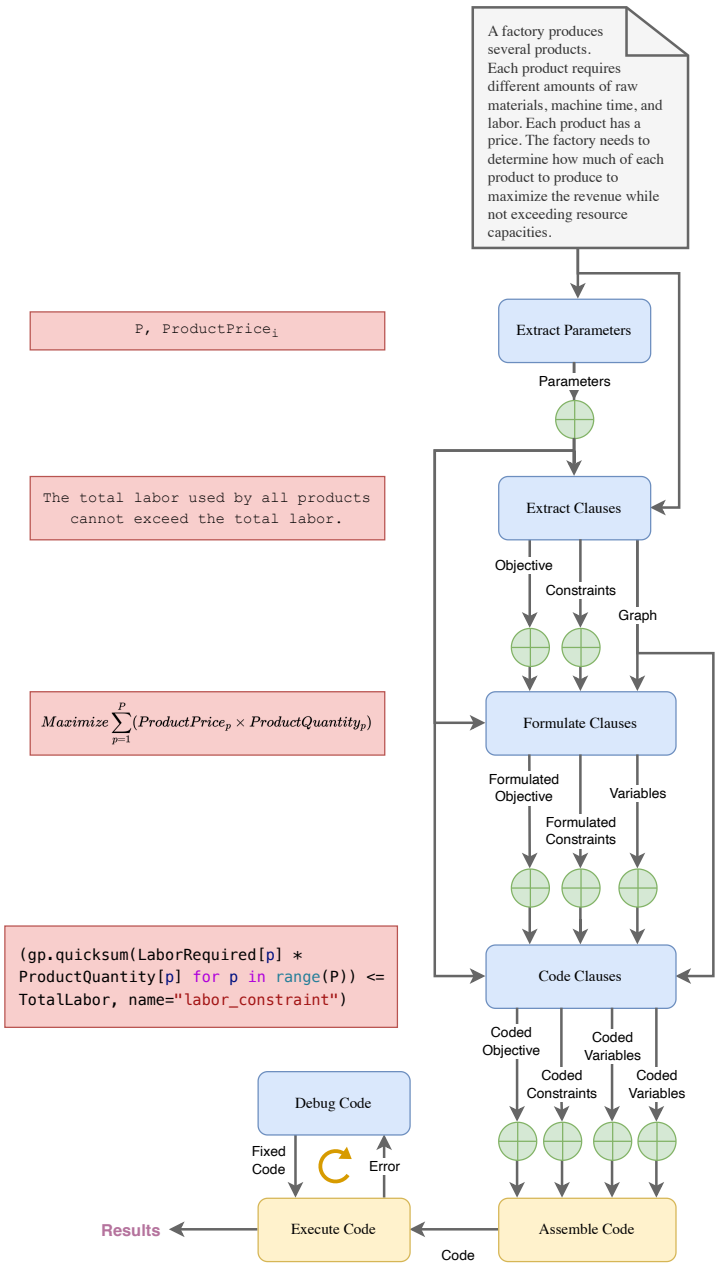


Figure 1 Optimus uses a sequential process with error correction (green circles) to model and solve optimization problems. It extracts the parameters and clauses, formulates them, generates the code for each, and finally synthesizes complete code and runs it. If the code raises an error, Optimus uses an iterative debugging loop to fix the code. Red boxes provide sample outputs of each step of the pipeline. Blue boxes indicate LLM-based components, and yellow boxes indicate deterministic components.

- **Background**: The background is a short string that explains the real-world context for the problem. Including this string in every prompt helps improve common sense reasoning.

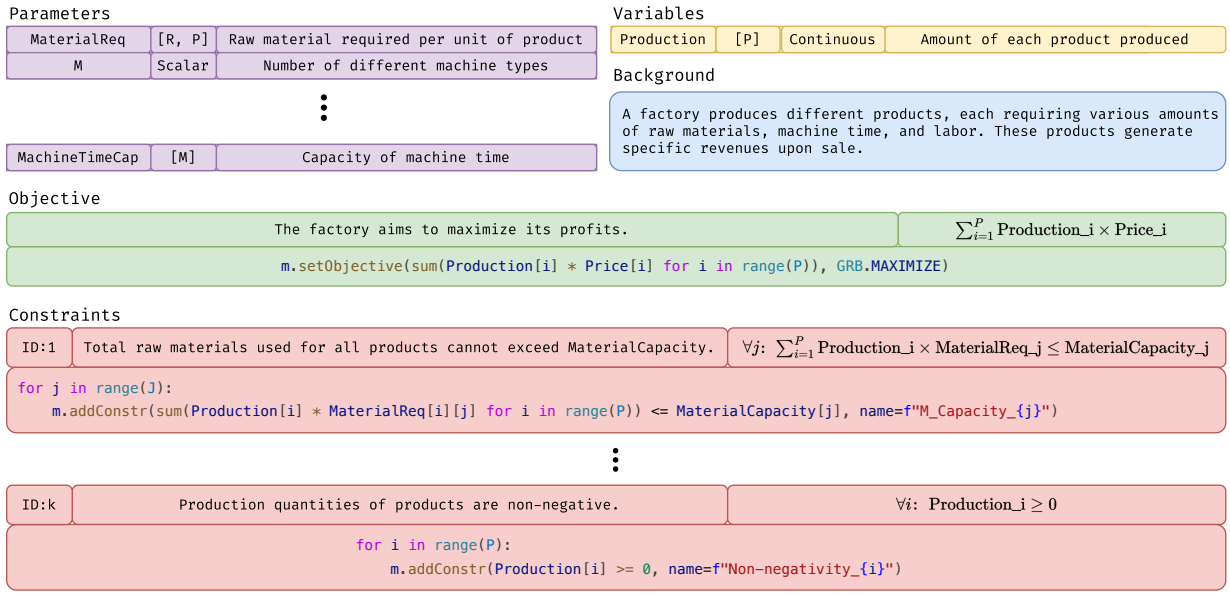


Figure 2 A completed state for a factory production optimization problem

- **Connection Graph**: The connection graph is a bipartite graph $G = (V, E)$ that links clauses to their associated parameters and variables. Formally, for every variable x_j (clause c_i) in our formulation we create a node v_{x_j} (v_{c_i}). In total, the connection graph consists of $n + m + 1$ nodes $v \in V$ corresponding to n variables, m constraints and the objective. We create an edge $e \in E$ between a variable node v_{x_j} and a clause node v_{c_i} if the variable participates in the clause (i.e., $(v_{x_j}, v_{c_i}) \in E$ if $a_{ij} \neq 0$). We build the connection graph iteratively by adding nodes and edges as more constraints and variables are formulated. This structure is utilized during the coding phase to ensure that only the relevant parameters and constraints are passed to the LLM.

The state is initialized as an empty object, and is completed step by step throughout the process. An example of the completed state for a factory optimization problem appears in Fig. 2.

4.2. Sample LLM Component

OptiMUS-0.3 uses LLMs as a flexible tool to perform various tasks for the overall system, including extracting parameters, modeling clauses, and coding the components (see Figure 1). In the interest of brevity, this section outlines the design of a typical LLM component with the OptiMUS framework. Each LLM component is governed by a natural language directive called a *prompt*. Within OptiMUS-0.3, each prompt includes the following key pieces of information:

- **Task Description:** We describe the specific task, e.g., "Your job is to extract natural language constraints for this paragraph defining an optimization problem".
- **Problem Context:** We include relevant information about the problem and the current solution in the prompt. During the formulation step, this involves the specific clause being addressed, along with

all parameters and variables defined so far. During the coding step, this involves the clause and its formulation, supplemented by related parameters and variables which are dynamically loaded using the connection graph.

- **Examples:** We include a fixed set of sample outputs for the task in the prompt to help the LLM perform In-Context Learning (Dong et al. 2024). For modules that output Python code, we also provide examples detailing the functionality of the gurobipy API.

For a comprehensive list of prompts used in OptiMUS-0.3 see our code[1]. Within the OptiMUS webapp, users can inspect and correct the output of every LLM-powered component of the system.

4.3. Error Correction

To build a trustworthy and reliable system using LLMs, it is important to mitigate the impact of LLM hallucinations. In the context of optimization modeling, an LLM might generate incorrect parameters, redundant mathematical constraints, or erroneous code. To address hallucinations, OptiMUS-0.3 uses two main error correction techniques: *reflective prompts* and *confidence-based user feedback*.

4.3.1. Reflective prompts LLMs can often identify and fix their mistakes by using reflective prompts (Shinn et al. 2023), a strategy by which an LLM is asked to reflect upon its output and decide if it needs to be corrected. We analyzed the most common types of mistakes made by LLMs on the development dataset at each step of the process and designed reflective prompts to address each one (see Appendix C for a full list of reflective prompts used in OptiMUS-0.3). Using these prompts significantly reduces the modeling error rate. For instance, Figure 3 shows an example where the reflexive prompt *Are units the same for both sides of this constraint?* can help correct a modeling error. The resulting improvement to OptiMUS is substantial (see Table 5), and substantiates the claim that domain-specific knowledge can be used to develop LLM applications that outperform general-purpose LLMs.

4.3.2. Confidence-based user/LLM feedback In the context of real-world optimization modeling, it is exceedingly rare for a natural-language problem description to correspond to an unambiguous MILP formulation. Rather, the process of optimization modeling generally involves many cycles of formulation and user feedback to arrive at a correct model for the problem. OptiMUS-0.3 simulates this process in order to detect errors and improve its solve rate. OptiMUS-0.3 can use a confidence-based reflective method to decide when to request help with its formulation, either from a user (if available) or another (more powerful) LLM.

First, following the methodology of existing work on confidence calibration in LLMs (Virk et al. 2024), we prompt the system to assess its confidence in its output on a scale of 1 to 5. If the confidence score is less than 5, OptiMUS-0.3 asks the user (or a stronger LLM) to help. The user can decide to either keep the

[1] included in supplementary materials

Are units the same for both sides of this constraint?

$(p_a + x_a) \cdot d_a \cdot (1 + e_a \cdot \frac{x_a}{p_a}) \leq m_a, \forall a \in A$?

... Left-hand side (LHS):

- $(p_a + x_a)$ represents the new price for article a, which is in euros (€).

- d_a represents the sales forecast (demand) for article a for the next twelve months at the current price, which is in units of the article.

- $(1 + e_a \cdot \frac{x_a}{p_a})$ is a unitless factor ...

Therefore, the unit of the left-hand side is: **euros (€) × units of the article**

Right-hand side (RHS):

- m_a represents the maximum production volume for article a, which is in units of the article (e.g., number of shirts, pants, etc.).

The unit of the right-hand side is: **units of the article**

... this inconsistency suggests an error in the formulation of Constraint 5. To correct this, we should ... here is the corrected constraint:

$$d_a \cdot (1 + e_a \cdot \frac{x_a}{p_a}) \leq m_a, \forall a \in A$$

Figure 3 OptiMUS-0.3 can fix its constraint modeling errors when prompted "*Are units the same for both sides of C?*"

output (e.g., constraint, code), remove it, or modify it. This confidence mechanism is applied across all three extraction and modeling modules: parameter extraction, variable identification, and constraint extraction. In the OptiMUS webapp (Section 6), this feature is used to highlight outputs that merit particular attention. In our experiments (Section 5), this feature is turned off by default to facilitate streamlined and reproducible testing without a user in the loop. See Fig. 4 for an example of how a weaker LLM (Llama-3-70B) can call on a stronger LLM (GPT-4o) when it is not confident in its solution.

4.3.3. Debug Code After the complete code is synthesized and executed by OptiMUS, any errors encountered during execution are passed to the LLM for debugging. At this stage, the LLM has the ability to modify any part of the code as needed. This step is particularly useful for resolving errors that arise from

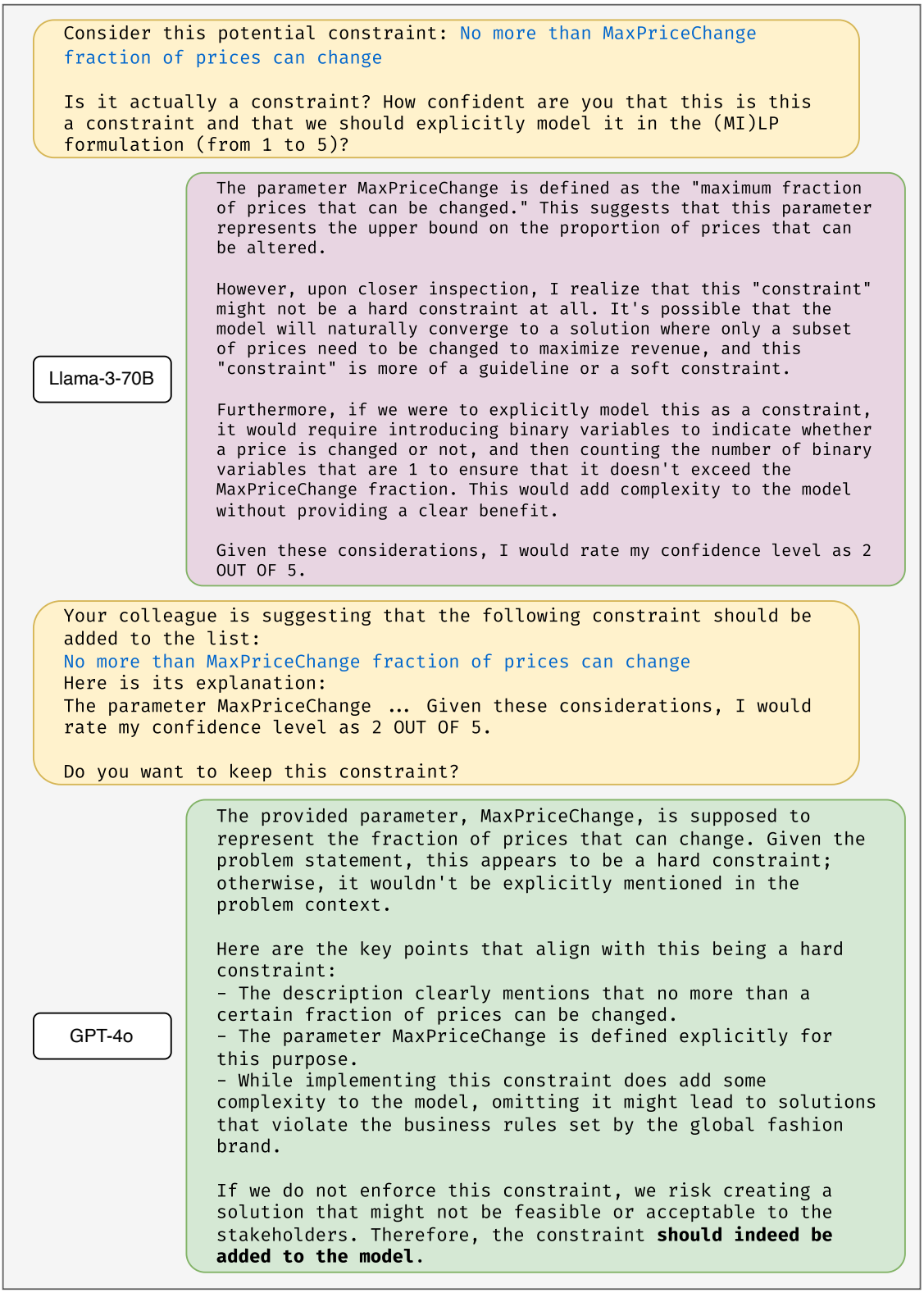


Figure 4 **GPT-4o can provide feedback when Llama-3-70B lacks the confidence to identify a constraint.**

inconsistencies across individual code snippets, such as slight variations in variable names or formatting. The debugging process is repeated iteratively until the code executes successfully, with a maximum of five attempts.

4.4. Structure Detection Agent

OptiMUS maintains a pool of optimization structures commonly used in optimization software. These structures can be specified explicitly in many modern optimization solver interfaces (Achterberg 2019, CPLEX User's Manual 1987, Gamrath et al. 2016), which can enhance problem-solving performance and simplify the code. Typical examples of structure include Special Ordered Set (SOS) (Beale and Forrest 1976), indicator variables, semi-continuous variables, and piecewise-linear constraints. At least one of these structures appears in approximately 10% of the NLP4LP dataset (see Appendix D). We expect these structures to be even more common, and thus important to exploit, in industrial applications.

To detect structure in a constraint (or variable), OptiMUS iterates through these structures and formats them into a structure detection prompt. Within each prompt, the LLM is provided with the structure description, explained by an example illustrating how the structure should be exploited. The LLM is asked to decide whether the structure is relevant to the existing formulation. Upon identifying the appropriate structure, the formulation is adjusted to highlight the problem structure. For instance, a set of constraints that indicate only one of a group of decision variables can be non-zero may be reformulated as type-1 SOS constraint. This structure is then conveyed to gurobi via the Python interfaces for special structures such as SOS constraints or indicator variables. This information can either be exploited within the branch and bound solver (e.g., using SOS constraints for branching rules) or allows the solver to automatically reformulate the structure into linear constraints (i.e., a big-M formulation for indicator constraints) (Gurobi Optimization 2023). While it may seem counter-intuitive to identify structure in a MILP only to have that structure re-formulated into linear constraints, in practice we found that automated methods within Gurobi can generate tighter formulations than OptiMUS alone (e.g., a better selection for big-M values). Figure 5 shows two instances where identifying SOS constraints and indicator variables leads to faster performance than a naive implementation. Full details of these examples are included in Appendix D. The structure detection agent is run during the formulate clauses stage of the algorithm.

OptiMUS also maintains a *problem structure* pool that enumerates combinatorial optimization problems with special structure that can be exploited in fast custom solvers (e.g., SAT, network flow, routing problems). For instance, the traveling salesman problem (TSP) can be solved much more efficiently using a specialized solver such as Concorde as opposed to general-purpose MILP solvers (Cook et al. 2011). OptiMUS iterates through this pool of special problem structures given an initial natural language description of the problem. When the optimization problem falls into one of the problem types in the structure pool, OptiMUS identifies the particular problem structure and suggests using a customized solver via a notification in the OptiMUS webapp.

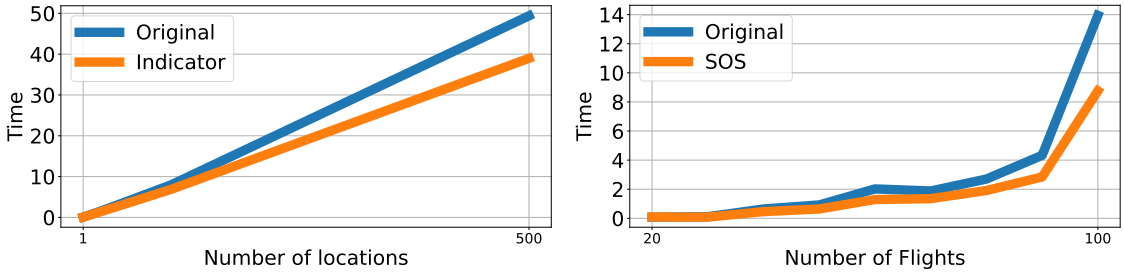


Figure 5 Impact of Structure Detection on OptiMUS Performance. (Left) Speedup of solving a facility location optimization problem with indicator variables with the structure identified to the solver (indicator) versus with OptiMUS modeling of indicator variable. (Right) Speedup of solving a flight assignment problem with SOS-constraints with the structure identified to the solver (SOS) versus with OptiMUS modeling of the SOS constraints. Full details of problem instances are included in Appendix D.

4.5. Advanced Optimization Coding Agent

For extremely large-scale optimization problems, optimization solvers are often embedded into a high-level optimization framework and called as a subroutine: examples include decomposition algorithms such as column generation, Benders decomposition, and cutting plane methods. To leverage variable and constraint structure in this large-scale context, the subproblem solver must invoke an advanced solver interface (for example, call-backs, model attribute query and analysis). While we leave the problem of developing a fully automated system that can perform advanced decomposition algorithms to future work, we have added several features to OptiMUS that advance this vision. OptiMUS implements a dedicated coding agent that exploits advanced solver functionality such as callbacks, and can iteratively call a solver within a simple variable or constraint sifting scheme (also known as delayed constraint generation). Neither of these modules requires explicitly reformulating the problem (e.g., generating a pricing problem within column generation), but allow for better computational performance than a naive implementation.

Similar to the structure detection agent, the optimization coding agent maintains a series of template prompts for this functionality. Within each prompt, the LLM is provided with the purpose of the template. We illustrate this functionality with an application to sifting, a simple version of constraint or column generation, in Section 4.5.1. The advanced coding agent runs during the Code Clauses stage of OptiMUS (see Fig. 1).

4.5.1. Sifting Sifting is a large-scale linear optimization framework initially proposed in (Bixby et al. 1992). Consider the primal linear optimization problem

$$\min_{x} \quad c^\top x \quad \text{subject to} \quad Ax = b, \quad x \geq 0,$$

where $A \in \mathbb{R}^{m \times n}$ and $n \gg m$. On the primal side, sifting combines the idea of primal simplex and column generation by solving a restricted master problem,

$$\min_{x_S} \quad c_S^\top x_S \quad \text{subject to} \quad A_S x_S = b, \quad x_S \geq 0,$$

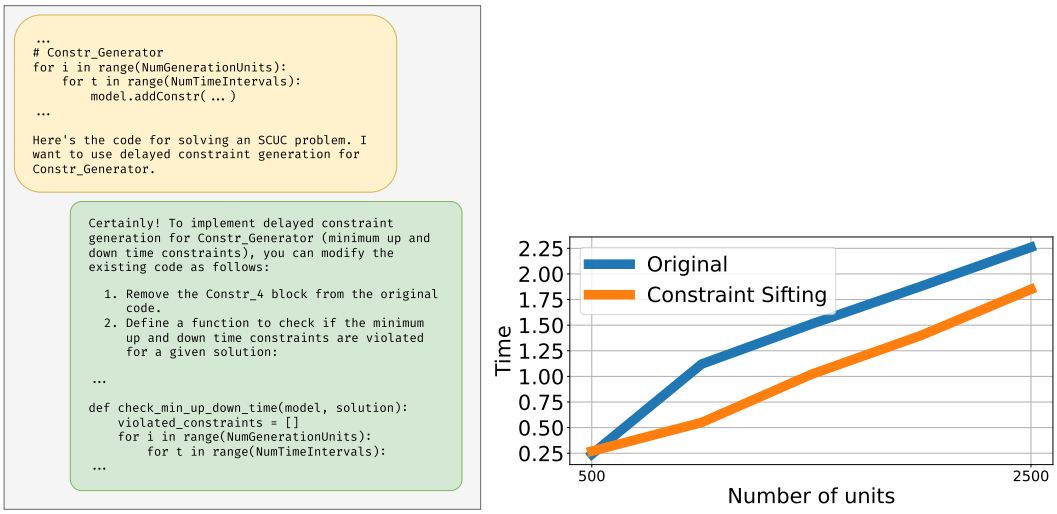


Figure 6 **Left: constraint sifting prompt. Right: performance plot for solving to a 5% gap.**

for a subset of columns $S \subseteq \{1, \ldots, n\}$. The restricted problem can be solved by any linear programming subroutine. Its dual variable y_S will be used to price out dual infeasible columns

$$I_S := \{j : c_j - A_j^\top y < 0\}.$$

Variable sifting works by iteratively updating $S \leftarrow S \cup I_S$ untill $S = \varnothing$. The idea can be generalized to the dual side when $m \gg n$, where a restricted problem contains a subset of rows. We provide an example of constraint sifting in OptiMUS in Fig. 6 on a security-constrained unit commitment problem. For details of the problem see Appendix E. Most of the constraints in the instance are inactive, making it a good candidate for sifting.

5. Experiments

In this section, we conduct a comprehensive evaluation of OptiMUS-0.3. We showcase the superior performance of OptiMUS-0.3 on both the NLP4LP dataset and two existing benchmark datasets for optimization modeling highlighting its strengths and weaknesses. An ablation study demonstrates the impact of different system components on our results, and a sensitivity analysis probes the internal dynamics of OptiMUS-0.3. We conclude this section by identifying failure cases and potential areas for further improvement.

5.1. Overall Performance

Baselines To evaluate the overall performance of OptiMUS, we compare it with standard prompting, Reflexion, and Chain-of-Experts (CoE) (Shinn et al. 2023, Xiao et al. 2023). Reflexion is the highest-performing general-purpose framework, and CoE is a recently proposed agentic framework for natural-language optimization modeling. We include self-reported results for two state-of-the-art auto-formulation methods based on fine-tuning LLMs specifically for optimization modeling: ORLM (Tang et al. 2024), and

LLMOPT (Jiang et al. 2024). We also include results from the initial conference publication of this work, which we denote as OptiMUS-0.2. We report results for the OptiMUS-0.3 framework with both GPT-4o (OpenAI 2023) and o3 (Jaech et al. 2024), an LLM that automatically includes more advanced reasoning at the expense of longer inference times. Due to the cost of running o3, we report performance with standard prompting from Song et al. (2026) and only run the OptiMUS framework on datasets where o3 alone does not already achieve near-perfect accuracy. In our experiments, we use the default API parameters for all LLM calls; exact model versions and inference settings are reported in Appendix H.

Evaluation Metrics and Datasets Three main metrics have been used to assess the accuracy of LP modeling tools in the literature: accuracy, compilation error (CE) rate, and runtime error (RE) rate. However, a method can generate an irrelevant short code that runs, or fix runtime and compilation errors by completely removing relevant sections of the code. Hence, we only compare the models' accuracy. Accuracy is defined as the number of instances correctly solved. An instance is considered as correctly solved only if 1) the code runs successfully, 2) the optimal value is correct (including infeasible instances), and 3) the optimal solution is correct. To check whether a generated solution matches the reference, we extract the numeric values of all decision variables from the solver output and compare them to the reference as multisets (unordered collections), making the check invariant to variable ordering. One limitation is that this requires the generated model to use the same variable space as the reference. A valid but differently-parameterized formulation (e.g., aggregate rather than per-unit variables) will produce a false negative, so condition 3 is a lower bound on true accuracy; condition 2 (optimal value match) serves as a softer complementary check. Not all benchmark datasets include ground-truth optimal solutions. For datasets that provide only optimal values (NL4OPT and IndustryOR), we check for correctness by assessing execution accuracy, counting an instance as correct if the code runs successfully and the optimal value matches the reported value. For NLP4LP, which includes both optimal values and solutions, we apply the full three-condition accuracy defined above.

In addition to the NLP4LP dataset introduced in this paper, we include results for two existing datasets for optimization modeling: NL4OPT (Ramamonjison et al 2023) and IndustryOR (Tang et al. 2024). The latter dataset includes 100 real-world case studies from eight different industries. To isolate core modeling and solving capabilities, all methods in Table 3 receive the same preprocessed input with parameter names identified. The OptiMUS system does not require preprocessed input: in fact, parameter extraction is the first pipeline step (Section 4), and the preprocessed input is equivalent to what OptiMUS's own extraction module produces. Rather, we provide preprocessed input to all methods to ensure a like-for-like comparison of modeling and solving ability.

Results are presented in Table 3. OptiMUS-0.3 outperforms all other direct prompting and agentic methods in all datasets by a large margin, beating the standard GPT-4o baseline by over 43% on the easy NLP4LP subset and by over 18% on the hard subset. This improvement persists even with more advanced reasoning

Table 3 Accuracy of auto-formulation methods on benchmark datasets. OptiMUS-0.2 requires an additional parameter conversion from the NLP4LP data files.

			NLP4LP			
	LLM	NL4OPT	Easy	Hard	Case Studies	IndustryOR
Methods based on direct prompting						
Standard	GPT-4o	47.3%	34.6%	20.3%	14.3%	28.0%
Standard	o3	97.3%[†]	62.7%	42.4%	28.6%	44.0%
Reflexion	GPT-4o	53.0%	35.8%	22.0%	14.3%	–
Methods based on fine-tuning LLMs						
LLMOPT	Qwen1.5-14B	93.0%[*]	83.8%[*‡]		–	46.0%[*]
ORLM	Deepseek-Math	85.7%[*]	85.7%	72.9%	0.0%	38.0%[*]
Methods based on agentic frameworks						
CoE	GPT-4o	–	64.2%	49.2%	–	–
OptiMUS-0.2	GPT-4o	78.8%	84.4%	26.4%	–	–
OptiMUS-0.3	GPT-4o	86.6%	78.2%	38.5%	42.9%	37.0%
OptiMUS-0.3	o3	–	79.9%	75.4%	28.6%	46.0%

[*] Self-reported from original paper; blanks indicate no results available.

[†] From Song et al. (2026).

[‡] Reported on a combined sample of the Easy and Hard subsets; per-subset breakdown not available.

models such as o3. This impressive performance improvement highlights the importance of modularity and structure compared to a single prompt to solve complex problems using LLMs.

The OptiMUS framework also remains competitive with fine-tuning approaches. With o3, OptiMUS comes within 6% of ORLM on the Easy subset and outperforms ORLM on the Hard, Case Studies, and IndustryOR benchmarks. The modular design of an agentic system thus closes much of the performance gap between direct prompting and finetuning, giving practitioners a simpler alternative that requires no retraining when better models are released. The improved performance with o3 over GPT-4o underscores this benefit directly, as updating OptiMUS to use a newer model requires only a change to the API call, whereas finetuning-based methods must repeat the full training procedure. This generalization gap is most stark on the case-study instances, where ORLM achieves 72.9% on the NLP4LP Hard subset but 0% on the case studies, while OptiMUS-0.3 (GPT-4o) achieves 42.9%, suggesting that fine-tuned models may not generalize to the complex, real-world MILP structure present in the case studies.

On Computation Time In Figure 7 we investigate the running time of the OptiMUS framework across 85 randomly selected instances of NLP4LP using GPT-4o and along different stages of the pipeline. The OptiMUS-0.3 framework takes no more than 350 seconds (median time 108 seconds) to model and solve any problem instance. For context, Tang et al. (2024) report that human subject matter experts require on average approximately 150 *minutes* to model and solve complex, real-world optimization problems of the kind collected in their IndustryOR benchmark; this figure should not be taken as representative of simpler,

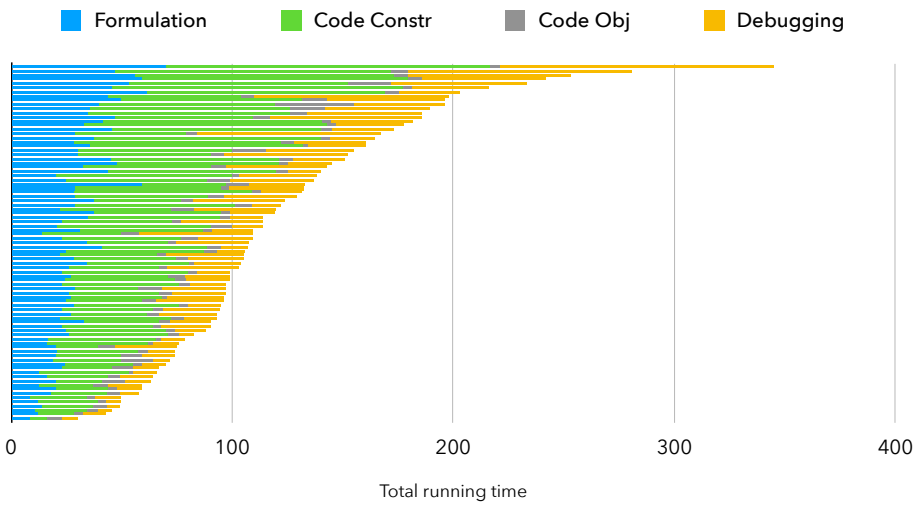


Figure 7 Distribution of running time (s) for different stages in the OptiMUS framework.

textbook-style instances. Coding and debugging represent the most time-consuming parts of the OptiMUS pipeline. Smaller LLM agents specialized to coding (e.g., codex) could further improve the computation time of the framework.

On Stochasticity in LLM Outputs LLM outputs are probabilistic, so calling the same model multiple times can produce different outputs. To evaluate the impact of this stochasticity on OptiMUS-0.3, we ran a stratified study: 15 randomly sampled easy instances and 15 hard instances, each run over 5 independent random seeds (150 total runs). This study is designed to measure *consistency* across seeds, not to re-estimate overall accuracy; authoritative accuracy figures are in Table 3.

Table 4 summarizes the results. Easy instances are highly stable: 11 of 15 always pass across all five seeds, 4 of 15 show one or two seed-level failures, and none always fail. Hard instances show substantial stochasticity: 10 of 15 switch between passing and failing across seeds, giving a pass@5 rate (at least one pass in five seeds) of 73%, well above the pass@1 rate of 35% for this sample. Repeated sampling therefore does recover correct solutions for many hard instances. No instance was ever classified as "incorrect" (code runs but wrong answer), so all failures are identifiable execution errors. The error-correction modules strongly stabilize easy instances but only partially dampen stochasticity on harder problems, where small variations in intermediate steps can tip the pipeline toward failure.

On Calibration A key feature of the OptiMUS-0.3 system is the confidence-based LLM feedback detailed in Section 4.3.2. To evaluate whether these confidence scores are well-calibrated, we collected 410 annotated outputs across all three pipeline modules (parameter extraction, variable identification, and constraint

Table 4 Stochasticity robustness: 15 easy and 15 hard instances run over 5 seeds each. *Always pass/fail*: consistent outcome across all 5 seeds. *Mixed*: outcome varies across seeds. pass@5: fraction of instances passing on at least one of 5 seeds.

Stratum	Instances	pass@5	Always pass	Mixed	Always fail
Easy	15	100%	11	4	0
Hard	15	73%	1	10	4

extraction), covering both simple NLP4LP instances and the seven real-world case studies. A doctoral student in operations research manually annotated each extracted item as correct or incorrect. We report results in a type-I/type-II framing.

Type-I errors (high confidence, item wrong) occurred in 0 out of 402 cases where the LLM assigned confidence ≥4/5; the system never reported high confidence on an incorrect extraction across any module. Type-II errors (low confidence, item correct) occurred in 4 out of 8 low-confidence predictions (score ≤3/5), giving an over-flagging rate of 50%. Among the 8 low-confidence items, accuracy was 25% at score 1/5, 67% at score 2/5, and 100% at score 3/5, confirming that lower scores are genuinely predictive of errors. This asymmetry is intentional, as falsely trusting a wrong extraction (type-I) is far more costly for a practitioner-facing tool than unnecessarily reviewing a correct one (type-II). The 0% type-I rate across 402 high-confidence predictions (spanning parameters, variables, and constraints on both simple and complex instances) provides strong evidence that the confidence mechanism is reliably conservative.

5.2. Ablation Study

Table 5 shows the impact of the choice of LLM on the performance of OptiMUS, broken down by its performance on the NLP4LP dataset. We evaluate the OptiMUS system with a leading closed-source LLM, GPT-4o, a reasoning model, o3, as well as a weaker open-source alternative, LLaMa-3.1-70B-Instruct (Meta AI 2024). As expected, the performance of the OptiMUS-0.3 framework degrades when less capable LLMs, such as LLaMa-3.1-70B-Instruct, are used instead of GPT-4o or o3, especially on hard modeling tasks. Optimization tasks need complex reasoning, and smaller models may be better suited for simpler tasks. However, the OptiMUS-0.3 framework with LLaMa-3.1-70B-Instruct outperforms GPT-4o with previous state-of-the-art prompting strategies, including Reflexion, on easy problems. We see that OptiMUS-0.3 is more robust to the quality of the underlying LLM, allowing weaker cheaper models to match the performance of stronger models on simple optimization modeling tasks.

Since the inception of the OptiMUS project, newer frontier models (e.g., OpenAI's GPT-5) have continued to improve code-and-reasoning capabilities, so the absolute accuracies in Table 5 should be interpreted as a point-in-time snapshot rather than a fixed ceiling. Moreover, because NL4OPT and NLP4LP are public benchmarks, it becomes increasingly difficult to fully rule out partial benchmark exposure for later models, a known issue in LLM evaluation, so our main takeaway is deliberately model-agnostic: the modular structure

of OptiMUS yields consistent gains using a strong general model (GPT-4o), a reasoning model (o3), and a weaker open model. We expect the same qualitative trend to hold for newer models, with diminishing improvements as benchmarks approach saturation.

Table 5 also shows the impact of different components of the OptiMUS framework on its performance. Each row removes one component from the full OptiMUS-0.3 pipeline while holding all others fixed, so the accuracy gap between that row and the full system quantifies the marginal contribution of that component.

Debugging (error correction on generated code) is the dominant driver of performance, reducing NLP4LP-Easy accuracy from 78.2% to 26.7% (–51.5pp) when removed. This effect is largest on hard instances, where LLMs frequently produce syntactically valid but semantically incorrect code; iterative solver feedback identifies and repairs these errors across multiple rounds. Extraction EC (error correction after clause extraction) accounts for the next largest contribution (–17.7pp, from 78.2% to 60.5%). It is especially important for hard instances, which involve multi-dimensional variables and complex indexing that the base extractor sometimes mishandles; crucially, missing constraints cannot be recovered later in the pipeline, so errors at this stage are final. Modeling EC (error correction after clause modeling) contributes a further –12.5pp (78.2% to 65.7%). Its effect is smaller than Extraction EC because many modeling errors can be recovered downstream in the debugging step. LLM Feedback contributes the smallest individual gain (–9.8pp, from 78.2% to 68.4%), catching residual cases where the solver returns a feasible but semantically incorrect solution. These effects are not independent (the full system's accuracy reflects their joint contribution), but the ablation rows make the marginal value of each component precise.

Table 6 further breaks down the impact of error correction methods on correct and incorrect items (i.e., constraints). In both constraint extraction and modeling, EC modules are able to fix a large fraction of errors without modifying most correct items. Note that we do not include the impact of the structure detection agent and advanced optimization coding agent as they do not impact the accuracy of the framework, but just the efficiency (i.e., runtime). For examples showcasing the impact of these components, see Appendix D.

Fig. 8 (left) shows the relation between the performance and the number of debugging steps for OptiMUS with both GPT-4o and LLaMa-3.1-70B-Instruct. In both models, the first few debugging steps improve performance, but further steps do not result in improvement.

5.3. Failure Cases

We next analyze the most common reasons why OptiMUS fails. We categorized failure cases manually via a qualitative coding process based in grounded theory (Charmaz 2006). We followed an open-coding process to analyze instances on which OptiMUS failed and tag the type of error. These initial codes were then grouped together into the following coarser categories:

- Extraction errors: OptiMUS extracts the wrong natural language constraints or objective (e.g., price is non-negative where price is a parameter), or fails to extract all of the constraints from the description.

Table 5 Ablation studies on OptiMUS-0.3

	NL4OPT	NLP4LP-Easy
Importance of Different Components		
w/o Debugging	73.2%	26.7%
w/o Extraction EC	86.7%	60.5%
w/o Modeling EC	83.8%	65.7%
w/o LLM Feedback	86.6%	68.4%
OptiMUS-0.3 (GPT-4o)	**86.6%**	**78.2%**
Performance with Different LLMs		
LLaMa3.1-70B-Instruct	70.4%	31.5%
GPT-4o	**86.6%**	78.2%
o3	–	**79.9%**

- Formulation errors: OptiMUS formulates a mathematical model that does not match the natural language constraint (e.g., it uses the wrong parameter to lower-bound a variable).
- Coding errors: OptiMUS does not always generate error-free code even after debugging. Coding errors most often occur when the LLM is confused by the language used (e.g. a runtime error due to accessing indices that do not exist in a parameter array).

As shown in Fig. 8, OptiMUS successfully extracts almost all the correct clauses for the easy dataset. This task is easy on the easy dataset because the easy instances are all LPs and involve only scalar values. However, for the hard instances — comprising MILP problems and multi-dimensional variables — parameter extraction and modeling are considerably more challenging than coding the resulting model. Note that the coding error rate is lower for the hard dataset because OptiMUS sometimes fails in modeling or formulating hard clauses, producing clauses that cannot be coded, which we do not include in the coding statistics.

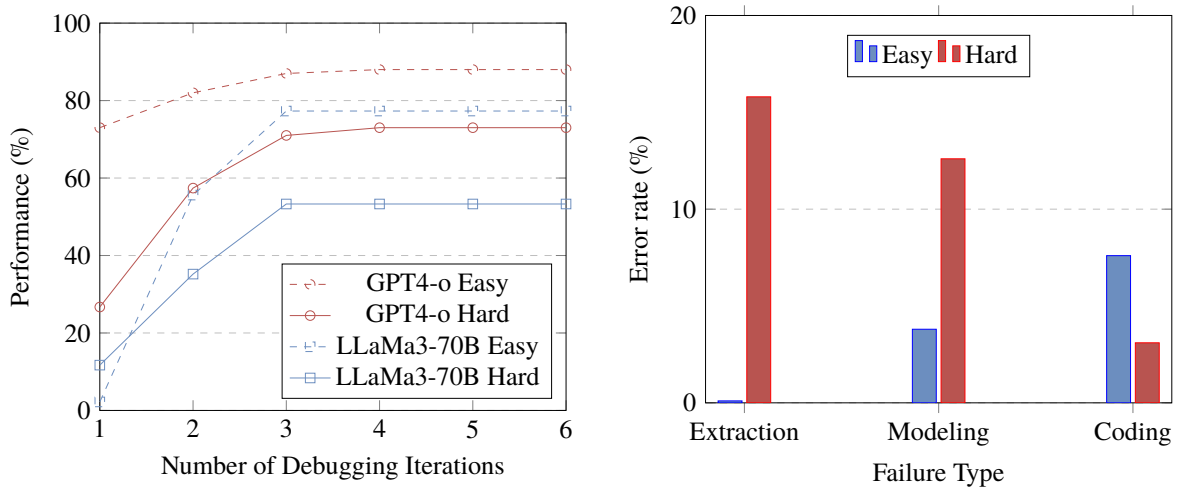


Figure 8 Left) Further debugging iterations improve performance. Right) For harder problems, most failures arise from clause extraction mistakes. For easier problems, most failures are due to coding errors.

Table 6 Error correction confusion matrices for constraint extraction (left) and modeling (right). Perfect performance is a diagonal matrix.

	Not Modified	Modified
Right	219	7
Wrong	9	41

	Not Modified	Modified
Right	231	2
Wrong	4	22

5.4. Scalability with Respect to Problem Complexity and Data Scale

We now provide fine-grainded analysis of the performance of OptiMUS on the seven case-study instances described in Section 3 (see Table 1), which test two of the scalability dimensions introduced in Section 1: *problem complexity* (rich combinatorial structure, multiple classes of integer and binary variables, logical constraints) and *data scale* (large-scale synthetic datasets generated to match the scales in each source paper). For each instance, we manually generate synthetic data and build and solve models according to the corresponding published paper. Table 7 reports the results. Although the version using o3 generates runnable code for five of the instances, only two of them are correct. On the other hand, GPT-4o generates runnable code for four of the instances, and three of them are correct. These results demonstrate the challenge that problem complexity and data scale pose for current LLM-based modeling systems, and establish a concrete benchmark for future work on these dimensions.

Table 7 Comparison of GPT-4o and o3 on real-world case studies. Gap $= |\text{Obj} - \text{Ref.Obj.}|/(|\text{Ref.Obj.}| + 1)$**;** ✓ **(0% gap) means correct;** × **means runtime error or infeasible.**

		GPT-4o		o3	
Instance ID	Ref. Obj.	Feasible	Gap	Feasible	Gap
355	201.4	✓	0%	✓	0%
356	851.7	✓	97.5%	✓	99.9%
357	746.9	×	×	×	×
358	1.0	✓	0%	×	×
359	570.0	×	×	✓	19.8%
360	820.0	✓	0%	✓	0%
361	1217.4	×	×	✓	99.7%
Correct		3 / 7		2 / 7	
Feasible		4 / 7		5 / 7	

Detailed analysis. To better understand the failed instances, we inspected each incorrect or infeasible instance. The errors fall into three broad categories. *Runtime errors* arise from implementation mistakes that prevent execution entirely: for instance 359, GPT-4o passes arc data stored as lists directly to Gurobi as variable keys, causing a type error since Gurobi requires hashable (tuple) indices; for instance 361, GPT-4o iterates over period indices starting from zero while the data keys begin at the actual starting period, producing a key error. *Wrong formulations* occur when the model runs but misrepresents the

problem structure: for instance 356, GPT-4o incorrectly defines excess as a binary per-student variable rather than a continuous per-(group, class) variable, and computes a quadratic deviation term incompatible with MIP solvers; for instance 361, o3 collapses the entire multi-period pool-management LP into a single scalar release decision per period, discarding the pool-state dynamics and multiorder-hit structure that drive the objective. *Incomplete or placeholder models* represent cases where the LLM fails to engage with the problem at all: for instance 356, o3 ignores all class and capacity data and minimizes a dummy group-index objective; for instance 357, GPT-4o adds contradictory balance constraints that render the model infeasible, while o3 completely ignores the optimization objective; for instance 358, o3 applies the DEA normalization constraint over all decision-making units instead of the evaluated unit, and leaves weight variables unbounded below, making the problem unbounded; for instance 359, o3 omits mandatory detour constraints for long block sides, traffic restrictions, and forbidden paths, finding a shorter but infeasible route.

Together, these failures highlight three key challenges for LLMs on real-world OR problems: correctly handling domain-specific data structures, faithfully translating complex multi-indexed formulations into code, and recognizing when a problem requires rich auxiliary structure beyond a simple objective and basic constraints.

Data scale. A key design feature of OptiMUS is that problem data is stored externally and referenced symbolically: the LLM sees only a structured description of parameter names and shapes, not the raw data values. As a result, the prompt length is independent of input data size. To validate this design, we compare OptiMUS against a *naive prompting baseline* that provides the full problem data as a JSON blob in the prompt alongside the description. We use the two case-study instances whose GPT-4o solutions are correct (Amazon Locker, instance 355; R-VRP, instance 360) and vary the input data size across different scales (i.e., by increasing the planning horizon for the locker problem and number of routes for VRP). For each (instance, scale) pair we generate 5 independent data instances (different random seeds), run both methods, and report *correct-formulation rate*: the fraction of seeds where the generated code runs and finds a high-quality objective. We use a 600-second solver budget for all runs and define success as matching the reference objective within 1%.

For instance 355 (Amazon Locker), OptiMUS maintains 100% correct-formulation rate at all 8 scale levels (data 3 KB–935 KB): the same generated code is applied to data of any size. Naive prompting degrades from 80% at T=5 to 40% at T=30, indicating genuine formulation errors as description and data diverge, and exceeds GPT-4o's 128K-token context window at $T \geq 75$ (data $\geq$245 KB), making it infeasible beyond that point. For instance 360 (R-VRP), OptiMUS achieves 100% correct-formulation rate at all 9 scale levels: it generates code once and applies it to data ranging from 2 KB to 385 KB. Naive prompting matches this

at small to medium scales (data 2 KB–190 KB), achieving 80% for two instances (one formulation error per seed out of five) and 100% at all other tested scales. At large scales (272–385 KB), naive prompting cannot generate code at all because the prompt exceeds GPT-4o's context window; OptiMUS continues to run unaffected.

Together, these results show that naive prompting has two distinct failure modes (formulation errors and context overflow at large scales), neither of which affects OptiMUS. The results further demonstrate that OptiMUS's data-separation architecture allows it to remain operational at data scales that are simply unreachable for naive prompting.

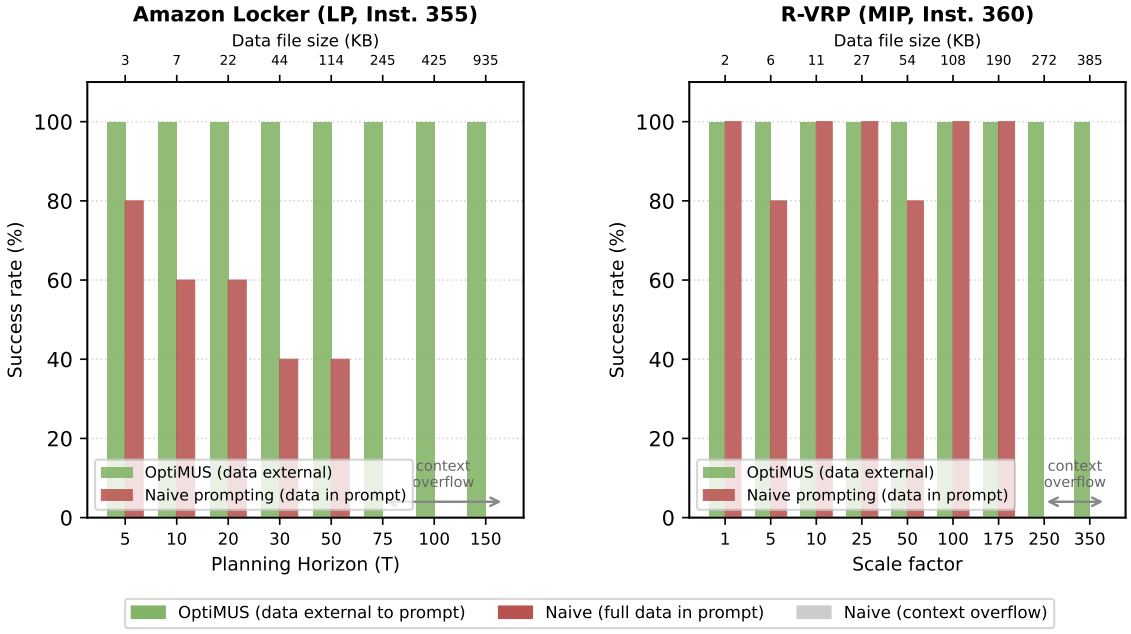


Figure 9 Correct-formulation rate (5 seeds per scale) for OptiMUS vs. naive prompting on instance 355 (LP, left) and instance 360 (MIP, right). Hatched bars indicate context overflow; OptiMUS, which stores data externally, is unaffected at all scales.

6. Web Application

As discussed in 1, the OptiMUS project aims to help optimization practitioners develop and maintain their models with ease, reducing development time and the risk of errors. To further these goals, we designed and built a user-centric web app with an intuitive interface that enables users to interact seamlessly with the system and leverage the advantages of large language models (LLMs). The webapp allows users to follow the overall process of modeling an optimization model including extracting parameters and clauses, modeling each clause, and finally coding it up. Importantly, users can correct any output of the framework, including natural language clauses, LaTeX model, and code, to provide better supervision over the entire process. By allowing users to observe and provide input throughout each step, the web app brings significant speed and



Figure 10 Sample view of the OptiMUS webapp during the coding phase in which individual clauses are translated to Python code. Users have the ability to see the confidence scores, correct the code directly in the panes to the right, or ask OptiMUS to regenerate code.

convenience to the modeling process while reducing the risk of errors. Figure 10 shows a sample model in the Coding stage. Further details of all the different stages and features incorporated in the webapp can be found in Appendix F. The webapp is publicly available at https://optimus-solver.com/.

7. Conclusion

How can we leverage LLMs to achieve complex goals? This paper interrogates this question in the domain of optimization and showcases the importance of modular structure: our ablation studies show that debugging alone accounts for nearly half of performance gains on hard problems, and modular architecture allows weaker models to match stronger models with naive prompting. We instantiate these principles in OptiMUS-0.3, a modular LLM-based agent that achieves state-of-the-art performance on NLP4LP, a new comprehensive dataset of challenging optimization problems, and scales to problems with long descriptions and complex real-world data. The primary lesson of this work is a design principle: in LLM-based optimization modeling, system architecture can enable weaker models to match stronger ones and agentic approaches to compete with fine-tuned specialist models without retraining costs.

OptiMUS is designed as a productivity tool for optimization practitioners: it accelerates the modeling and implementation workflow for users who already understand the problem domain, in the same way that GitHub Copilot accelerates software development for engineers who already know how to code. Real-world optimization problems are often complex and multifaceted. Developing LLM-based solutions for these problems requires domain-specific considerations, including integrating existing optimization techniques to

leverage problem structure. We are at the early stages of this research, but anticipate significant developments that will enable these systems to address more complex, industrial-level problems. It is interesting to notice that the challenge of using AI for an applied domain is much larger in safety-critical domains such as self-driving, which demand extremely high accuracy, than in domains where AI can function as an assistant and where answers can be checked by humans, as in theorem-proving or optimization. Here, AI systems with moderate accuracy can still usefully augment human effort.

Future directions. The OptiMUS system presented here serves as a prototype for more powerful AI-assisted optimization modeling, and a step towards making classical optimization tools accessible to a broader range of users. Much more work remains to make such a system reliable enough to reduce the expertise required to deploy optimization in practice. While some improvements can be expected as language models grow more powerful, we expect that substantial breakthroughs by the optimization community will be required to address many of the following challenges:

- **Reliability:** Optimization algorithms generally offer guaranteed reliability. How can a system like OptiMUS reliably solve large-scale problems using unreliable LLMs that suffer hallucinations, overconfidence, and instability? What kinds of guarantees can be made, and how expensive might these be? How should developers of such a system assess whether the system has succeeded in producing a correct model or not?
- **Trust:** A system like OptiMUS should help domain experts understand the model that was selected and decide whether it meets their needs. How should the system communicate a model back to the human user to facilitate effective feedback about the deficiencies in the model, or trust in a model that is correct? How should the system integrate this feedback? How can the system developer assess the effectiveness of this feedback and improve the system to enable quicker and easier feedback cycles?
- **Ambiguity:** Most optimization problems specified in natural language do not correspond unambiguously to a single optimization model. Rather, the process of formalizing the objective and constraints reduces the vagueness so that it is possible to distinguish a solution from any other proposal. A system like OptiMUS should help decision-makers think through their goals (which may be vague) to choose a model formulation by clarifying implicit requirements, eliciting the relative importance of different objectives, and searching internal or external databases for relevant data.
- **Fast Solvers:** Rapid decisions require a fast solver: possibly a well-chosen solver for the problem, or possibly even a custom solver written for the problem. A system like OptiMUS should be able to choose between available heuristics and parameters for provably optimal solvers using all information available about the problem to determine a solution method that meets runtime and accuracy requirements, or even to write a custom solver when it is warranted. To choose or write the best solver, we expect that the system will likely make use of natural-language information about the problem setting (using an LLM) as well as structured information about problem parameters (using a graph neural network).

- **Larger datasets.** All of these research directions depend on the availability of appropriate datasets to develop, hone, and test new ideas. Most of these directions require a dataset of optimization problems consisting of a natural-language description coupled with problem data and solution code. While there are many excellent datasets for developing optimization algorithms, these are generally not associated with natural-language descriptions; and problems associated with natural language descriptions are generally either small (e.g., textbook problems) or lack associated problem data and solutions (e.g., published papers on applications of MIPs) as these elements are often proprietary. Semi-synthetic approaches to generating larger datasets are promising (Tang et al. 2024), but creating MIP problems that are large-scale, realistic, and feasible is a challenge. Curating a larger dataset of optimization problems could be a key enabler driving further research and development in this area.
- **Beyond MILPs.** OptiMUS currently uses gurobipy as a modeling language. We made this choice as the wealth of online examples allows LLMs to successfully model many problems in gurobipy. However, many optimization problems are best expressed in a different framework: for example, Concorde for traveling salesman problems (Cook et al. 2011), cvxpy for control problems (Diamond and Boyd 2016), or minizinc for constraint programming problems (Nethercote et al. 2007). Integrating these alternative modeling languages and identifying, for a given problem, which modeling language to use (or whether the problem is likely to be successfully modeled in the OptiMUS framework at all) is an important future direction.

Acknowledgments

AA, WG, CL, and MU gratefully acknowledge support from the National Science Foundation (NSF) Award IIS-2233762, the Office of Naval Research (ONR) Awards N000142212825, N000142412306, and N000142312203, the Alfred P. Sloan Foundation, and from IBM Research as a founding member of Stanford Institute for Human-centered Artificial Intelligence (HAI). HB thanks the Aker Scholarship Foundation for financial support. ST gratefully acknowledges support from the Air Force Office of Scientific Research (AFOSR) under Grant FA9550-23-1-0251 and in part by the Office of Naval Research under Grant N00014-24-1-2164. This manuscript was improved by feedback from anonymous referees.

References

Jesper Aastrup and Herbert Kotzab. Forty years of out-of-stock research — and shelves are still empty. *The International Review of Retail, Distribution and Consumer Research*, 20(1):147–164, 2010.

Tobias Achterberg. What's new in gurobi 9.0. *Webinar Talk url: https://www. gurobi. com/wp-content/uploads/2019/12/Gurobi-90-Overview-Webinar-Slides-1. pdf*, 2019.

Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 577–596. PMLR, 21–27 Jul 2024.

Alibaba Cloud. Alibaba cloud mindopt copilot, 2022. URL `https://opt.alibabacloud.com/chat`.

Nicolás Astorga, Tennison Liu, Yuanzhang Xiao, and Mihaela van der Schaar. Autoformulation of mathematical optimization models using llms. *arXiv preprint arXiv:2411.01679*, 2024.

EML Beale and John JH Forrest. Global optimization using special ordered sets. *Mathematical Programming*, 10: 52–69, 1976.

Federico Bertero, Manuela Cerdeiro, Guillermo A Durán, and Nazareno A Faillace Mullen. Developing optimization tools for municipal solid waste collection in the argentine city of berazategui. *INFORMS Journal on Applied Analytics*, 53(6):451–464, 2023.

Dimitris Bertsimas and John N Tsitsiklis. *Introduction to linear optimization*, volume 6. Athena scientific Belmont, MA, 1997.

Robert E Bixby, John W Gregory, Irvin J Lustig, Roy E Marsten, and David F Shanno. Very large-scale linear programming: A case study in combining interior point and simplex methods. *Operations research*, 40(5):885–897, 1992.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.

Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, March 2004.

Kathy Charmaz. *Constructing grounded theory: A practical guide through qualitative analysis*. Sage, New York, NY, 2006.

Hao Chen, Gonzalo E Constante-Flores, and Can Li. Diagnosing infeasible optimization problems using large language models. *arXiv preprint arXiv:2308.12923*, 2023.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling language modeling with pathways, 2022.

William J Cook, David L Applegate, Robert E Bixby, and Vasek Chvatal. *The traveling salesman problem: a computational study*. Princeton university press, 2011.

CPLEX User's Manual. IBM ILOG CPLEX optimization studio. *Version*, 12(1987-2018):1, 1987.

Steven Diamond and Stephen Boyd. CVXPY: A python-embedded modeling language for convex optimization. *The Journal of Machine Learning Research*, 17(1):2909–2913, 2016.

Geri L Dimas, Malak El Khalkhali, Alex Bender, Kayse Lee Maass, Renata A Konrad, Jeffrey S Blom, Joe Zhu, and Andrew C Trapp. Estimating effectiveness of identifying human trafficking via data envelopment analysis. *INFORMS Journal on Applied Analytics*, 53(6):408–424, 2023.

Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Tianyu Liu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. A survey on in-context learning, 2024. URL https://arxiv.org/abs/2301.00234.

Gerald Gamrath, Timo Berthold, Stefan Heinz, and Michael Winkler. *Structure-based primal heuristics for mixed integer programming*. Springer, 2016.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023.

Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, et al. Miplib 2017: data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation*, 13(3):443–490, 2021.

Gurobi Optimization. 2023 state of mathematical optimization report, 2023. URL https://www.gurobi.com/resources/report-state-of-mathematical-optimization-2023/.

Jesse Holzer, Carleton Coffrin, Christopher DeMarco, Ray Duthu, Stephen Elbert, Brent Eldridge, Tarek Elgindy, Scott Greene, Nongchao Guo, Elaine Hale, Bernard Lesieutre, Terrence Mak, Colin McMillan, Hans Mittelmann, Hyungseon Oh, Richard O'Neill, Thomas Overbye, Bryan Palmintier, Farnaz Safdarian, Ahmad Tbaileh, Pascal Van Hentenryck, Arun Veeramany, and Jessica Wert. Grid optimization competition challenge 3 problem formulation. https://gocompetition.energy.gov/sites/default/files/Challenge3_Problem_Formulation_20230126.pdf, 2023. Accessed: Access Date.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

Caigao Jiang, Xiang Shu, Hong Qian, Xingyu Lu, Jun Zhou, Aimin Zhou, and Yang Yu. Llmopt: Learning to define and solve general optimization problems from scratch. *arXiv preprint arXiv:2410.13213*, 2024.

Jakob Krarup and Peter Mark Pruzan. The simple plant location problem: Survey and synthesis. *European journal of operational research*, 12(36-81):41, 1983.

Connor Lawless, Jakob Schoeffer, Lindy Le, Kael Rowan, Shilad Sen, Cristina St. Hill, Jina Suh, and Bahareh Sarrafzadeh. "I want it that way": Enabling interactive decision support using large language models and constraint programming. *ACM Transactions on Interactive Intelligent Systems*, 14(3):1–33, 2024.

Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models, 2024.

Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large language models for supply chain optimization. *arXiv preprint arXiv:2307.03875*, 2023.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023.

Steven H. Low. Convex relaxation of optimal power flow: A tutorial. In *2013 IREP Symposium Bulk Power System Dynamics and Control - IX Optimization, Security and Control of the Emerging Power Grid*, pages 1–15, 2013. doi: 10.1109/IREP.2013.6629391.

Zhi-Quan Luo, Wing-Kin Ma, Anthony Man-Cho So, Yinyu Ye, and Shuzhong Zhang. Semidefinite relaxation of quadratic optimization problems. *IEEE Signal Processing Magazine*, 27(3):20–34, 2010.

Meta AI. Meta LLaMA 3 Announcement, 2024. URL https://ai.meta.com/blog/meta-llama-3/. [Accessed: October 27, 2024].

Ciamac C Moallemi and Utkarsh Patange. Hybrid scheduling with mixed-integer programming at columbia business school. *INFORMS Journal on Applied Analytics*, 54(3):222–240, 2024.

Dritan Nace. Lecture notes in linear programming modeling, 2020. URL https://www.hds.utc.fr/~dnace/dokuwiki/_media/fr/lp-modelling_upt_p2021.pdf.

Nicholas Nethercote, Peter J Stuckey, Ralph Becket, Sebastian Brand, Gregory J Duck, and Guido Tack. Minizinc: Towards a standard cp modelling language. In *International Conference on Principles and Practice of Constraint Programming*, pages 529–543. Springer, 2007.

OpenAI. Gpt-4 technical report, 2023.

Yonatan Oren, Nicole Meister, Niladri Chatterji, Faisal Ladhak, and Tatsunori B. Hashimoto. Proving test set contamination in black box language models, 2023. URL https://arxiv.org/abs/2310.17623.

PapersOwl. Free Plagiarism Checker, 2024. URL https://papersowl.com/free-plagiarism-checker. [Accessed: Date of Access, e.g., October 27, 2024].

Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. ART: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*, 2023.

. Ramamonjison et al. Augmenting operations research with auto-formulation of optimization models from problem descriptions. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 29–62, Abu Dhabi, UAE, December 2022. Association for Computational Linguistics. URL https://aclanthology.org/2022.emnlp-industry.4.

. Ramamonjison et al. NL4Opt competition: Formulating optimization problems based on their natural language descriptions, 2023. URL https://arxiv.org/abs/2303.08233.

Tomáš Roubíček. *Relaxation in optimization theory and variational calculus*, volume 4. Walter de Gruyter GmbH & Co KG, 2020.

Soroush Saghafian, Garrett Austin, and Stephen J Traub. Operations research/management contributions to emergency department patient flow optimization: Review and research prospects. *IIE Transactions on Healthcare Systems Engineering*, 5(2):101–123, 2015.

Samyukta Sethuraman, Ankur Bansal, Setareh Mardan, Mauricio GC Resende, and Timothy L Jacobs. Amazon locker capacity management. *INFORMS Journal on Applied Analytics*, 54(6):455–470, 2024.

Rabia Shakoor, Mohammad Yusri Hassan, Abdur Raheem, and Yuan-Kang Wu. Wake effect modeling: A review of wind farm layout optimization using Jensen's model. *Renewable and Sustainable Energy Reviews*, 58:1048–1059, 2016.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.

Yang Song, Anoushka Vyas, Zirui Wei, Sina Khoshfetrat Pakazad, Henrik Ohlsson, and Graham Neubig. Nemo: Execution-aware optimization modeling via autonomous coding agents. *arXiv preprint arXiv:2601.21372*, 2026.

Zhengyang Tang, Chenyu Huang, Xin Zheng, Shixi Hu, Zizhuo Wang, Dongdong Ge, and Benyou Wang. Orlm: Training large language models for optimization modeling, 2024. URL `https://arxiv.org/abs/2405.17743`.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Niklas Tuma, Manuel Ostermeier, and Alexander Hübner. Optimal transportation planning for a do-it-yourself retailer with a zone-based tariff. *INFORMS Journal on Applied Analytics*, 54(4):312–328, 2024.

Yuvraj Virk, Premkumar Devanbu, and Toufique Ahmed. Enhancing trust in llm-generated code summaries with calibrated confidence scores. *arXiv preprint arXiv:2404.19318*, 2024.

Yang Wang, Tong Wang, Xiaoqing Wang, Yuming Deng, and Lei Cao. Data-driven order fulfillment consolidation for online grocery retailing. *INFORMS Journal on Applied Analytics*, 54(3):211–221, 2024.

Segev Wasserkrug, Leonard Boussioux, Dick den Hertog, Farzaneh Mirzazadeh, Ilker Birbil, Jannis Kurtz, and Donato Maragno. From large language models and optimization to decision optimization copilot: A research manifesto. *arXiv preprint arXiv:2402.16269*, 2024.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought prompting elicits reasoning in large language models, 2023.

H Paul Williams. *Model building in mathematical programming*. John Wiley & Sons, 2013.

Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, et al. Chain-of-Experts: When LLMs meet complex operations research problems. In *The Twelfth International Conference on Learning Representations*, 2023.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers, 2023.

Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Wei Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. Optibench meets resocratic: Measure and improve llms for optimization modeling. *arXiv preprint arXiv:2407.09887*, 2024.

Enjian Yao, Tong Liu, Tianwei Lu, and Yang Yang. Optimization of electric vehicle scheduling with multiple vehicle types in public transport. *Sustainable Cities and Society*, 52:101862, 2020.

Fariba Zohrizadeh, Cedric Josz, Ming Jin, Ramtin Madani, Javad Lavaei, and Somayeh Sojoudi. A survey on conic relaxations of optimal power flow problem. *European journal of operational research*, 287(2):391–409, 2020.

Appendix

A. Detailed Solution Statistics

Table 8 GPT-4o per-instance results on NLP4LP (instances 1–354). † = Hard (instances 290–354). ✓ = correct, ✗ = failed, infeas = infeasible/unbounded.

ID	Status	ID	Status	ID	Status	ID	Status	ID	Status	ID	Status	ID	Status	ID	Status
1	✓	46	✓	91	✓	136	✓	181	infeas	226	✓	271	✓	316†	✗
2	✓	47	✓	92	✓	137	✓	182	✓	227	✗	272	✓	317†	infeas
3	infeas	48	✗	93	✓	138	✓	183	✓	228	✓	273	✓	318†	✗
4	✗	49	✓	94	✓	139	✗	184	infeas	229	✓	274	✓	319†	✓
5	✗	50	✓	95	✓	140	✗	185	✗	230	✗	275	✓	320†	✓
6	✗	51	infeas	96	✓	141	✗	186	✗	231	✗	276	infeas	321†	✓
7	✗	52	✓	97	✗	142	✓	187	✓	232	✗	277	infeas	322†	✗
8	✓	53	✗	98	✗	143	✓	188	✗	233	✓	278	✓	323†	✓
9	✓	54	✓	99	✓	144	✓	189	✓	234	✓	279	✓	324†	✓
10	✓	55	✓	100	✓	145	✗	190	✓	235	✓	280	✓	325†	✓
11	✓	56	✓	101	✓	146	✓	191	✓	236	✓	281	✓	326†	infeas
12	✓	57	infeas	102	✓	147	✓	192	✓	237	infeas	282	✓	327†	✓
13	✓	58	infeas	103	infeas	148	✓	193	✓	238	infeas	283	✓	328†	✓
14	✗	59	✓	104	✓	149	✓	194	✓	239	✓	284	✓	329†	✓
15	✓	60	✓	105	✓	150	✗	195	✓	240	infeas	285	✓	330†	✗
16	✓	61	✗	106	✓	151	✗	196	✓	241	✓	286	✓	331†	✗
17	✓	62	✓	107	✓	152	✗	197	✓	242	✓	287	✓	332†	✗
18	✓	63	✓	108	✓	153	✓	198	✓	243	✗	288	✓	333†	✗
19	✓	64	✓	109	✓	154	✓	199	✓	244	infeas	289	✓	334†	✗
20	✓	65	✓	110	✓	155	✓	200	✓	245	✗	290†	✓	335†	✗
21	✓	66	✓	111	✓	156	✗	201	✓	246	✓	291†	✓	336†	✗
22	✓	67	✓	112	✓	157	✗	202	✓	247	✓	292†	✓	337†	✗
23	✓	68	infeas	113	✓	158	✗	203	✓	248	✓	293†	✗	338†	✗
24	✓	69	✗	114	✓	159	✓	204	✓	249	✓	294†	✓	339†	✗
25	✓	70	✓	115	✓	160	✗	205	✓	250	✗	295†	✓	340†	✗
26	✓	71	✗	116	✓	161	✓	206	✓	251	infeas	296†	✓	341†	✗
27	✓	72	✗	117	✓	162	✗	207	✓	252	✓	297†	infeas	342†	✗
28	infeas	73	✗	118	✗	163	✓	208	✓	253	✗	298†	✓	343†	✗
29	✓	74	✓	119	✓	164	✓	209	✓	254	✓	299†	✗	344†	✗
30	✓	75	✓	120	✗	165	✓	210	✓	255	✓	300†	✓	345†	✗
31	✓	76	✓	121	✓	166	✓	211	✓	256	✓	301†	✗	346†	✗
32	✓	77	infeas	122	✓	167	✓	212	✓	257	✗	302†	✓	347†	✗
33	✓	78	✗	123	infeas	168	✗	213	✗	258	✓	303†	✗	348†	✗
34	✓	79	infeas	124	✓	169	✓	214	✗	259	✓	304†	✗	349†	✗
35	✓	80	✓	125	✓	170	✗	215	✓	260	✗	305†	✗	350†	✗
36	✓	81	✓	126	infeas	171	infeas	216	✗	261	infeas	306†	✗	351†	✗
37	✓	82	✓	127	✓	172	✗	217	✗	262	✗	307†	✓	352†	✗
38	✓	83	infeas	128	✓	173	✓	218	✓	263	✓	308†	✗	353†	✗
39	✓	84	✓	129	✓	174	✓	219	infeas	264	✓	309†	✗	354†	✗
40	✓	85	✓	130	✓	175	✓	220	✓	265	✓	310†	✗		
41	✓	86	✓	131	✓	176	✗	221	✗	266	infeas	311†	infeas		
42	✗	87	✗	132	✗	177	✗	222	✓	267	infeas	312†	infeas		
43	✓	88	✗	133	✓	178	✗	223	✓	268	✗	313†	✗		
44	✗	89	✓	134	✓	179	infeas	224	infeas	269	✓	314†	✗		
45	✓	90	✗	135	infeas	180	✓	225	✗	270	✗	315†	infeas		

B. Terms and Definitions

Table 10: Definitions of Terms Used in OptiMUS-0.3

Term: Clause	
Definition:	A constraint or objective within the optimization problem.
How We Use Them:	We refer to constraints and objectives as clauses to simplify terminology.
Example:	The constraint $x + y \leq 10$ is a clause.

Term: Connection graph	
Definition:	A graph that records which variables and parameters appear in each constraint.
How We Use Them:	Used to ensure consistency of formulations and focus the LLM on relevant context.
Example:	Constraint $C1$ connects to variables x and y in the connection graph.

Term: State	
Definition:	The collection of all parameters, clauses, variables, and background information managed and modified during the solution process.
How We Use Them:	Saved and updated in JSON format throughout the problem-solving steps.
Example:	The state includes parameters like demand, variables like production quantity, and clauses like capacity constraints.

Term: Reflective prompts	
Definition:	Prompts designed to encourage the LLM to reflect on and correct its own mistakes.
How We Use Them:	Used to reduce modeling errors by having the LLM check and fix its outputs.
Example:	Asking ”Are units the same for both sides of constraint C?”

Term: Confidence-based user feedback	
Definition:	A method where the LLM assesses its confidence and, if low, requests help from the user or a stronger LLM.
How We Use Them:	To improve the accuracy of the model when the LLM is unsure about its outputs.
Example:	The LLM says ”I am not confident about constraint C” and asks the user to verify.

Term: Parameters	
Definition:	Known quantities in the optimization problem, each with a symbol, shape, and definition.
How We Use Them:	Extracted from the problem description and used in clause formulations.
Example:	The cost per unit, denoted as c, is a parameter.

Term: Variables	
Definition:	Unknown quantities to be determined in the optimization problem, each with a symbol, shape, definition, and type.
How We Use Them:	Defined during clause formulation and used in constraints and objectives.
Example:	The production quantity x is a variable.

Term: Background	
Definition:	A short string that explains the real-world context of the problem.
How We Use Them:	Included in every prompt to improve common sense reasoning.
Example:	”This problem involves optimizing factory production.”

Term: Error Correction	
Definition:	Techniques used to mitigate the impact of LLM hallucinations and correct errors.
How We Use Them:	Using reflective prompts and confidence-based user feedback to improve reliability.
Example:	Correcting a misidentified parameter by prompting ”Is the value of P known or not?”

Term: Formulate Clauses	
Definition:	A process step where clauses are mathematically formulated, and variables and auxiliary constraints are defined.
How We Use Them:	To generate the mathematical representation of constraints and objectives.
Example:	Formulating "Total production must meet demand" as $x \geq d$.

Term: Extract Parameters	
Definition:	A process step where parameters are extracted from the problem description.
How We Use Them:	To identify all known quantities needed for the optimization problem.
Example:	Extracting "demand" as a parameter from the description.

Term: Correct Errors	
Definition:	A process step where errors in the parameters, clauses, and variables are corrected.
How We Use Them:	To ensure accuracy of the extracted and formulated components.
Example:	Correcting a parameter's shape if it was misidentified.

Term: Extract Targets	
Definition:	A process step where constraints, objective, and connection graph are extracted.
How We Use Them:	To identify the clauses and their relationships to parameters and variables.
Example:	Extracting the constraint "Production capacity cannot exceed limit."

Term: Code Targets	
Definition:	A process step where code snippets for all parameters, clauses, and variables are generated.
How We Use Them:	To assemble these snippets into a single runnable code file.
Example:	Generating code to define variables in a solver.

Term: Assemble Code	
Definition:	A process step where all code snippets are combined into a single code file.
How We Use Them:	To produce executable code for solving the optimization problem.
Example:	Combining parameter definitions, variable declarations, and constraints into one script.

Term: Debug	
Definition:	A process step where code is iteratively executed and errors are fixed until the desired results are achieved.
How We Use Them:	To correct syntax or runtime errors in the generated code.
Example:	Fixing a variable name mismatch that caused a runtime error.

C. Reflexive Prompts

The reflective prompts used by OptiMUS-0.3 are as follows:

Table 9 O3 per-instance results on NLP4LP (instances 1–354). † = Hard (instances 290–354). ✓ = correct, ✗ = failed, infeas = infeasible/unbounded.

ID	Status	ID	Status	ID	Status	ID	Status	ID	Status	ID	Status	ID	Status	ID	Status
1	✓	46	✓	91	✓	136	✓	181	infeas	226	✓	271	✓	316†	✗
2	✓	47	✓	92	✓	137	✓	182	✓	227	✗	272	✓	317†	infeas
3	infeas	48	✗	93	✓	138	✓	183	✓	228	✓	273	✓	318†	✗
4	✗	49	✓	94	✓	139	✗	184	infeas	229	✓	274	✓	319†	✓
5	✓	50	✓	95	✓	140	✗	185	✗	230	✗	275	✓	320†	✗
6	✗	51	infeas	96	✗	141	✗	186	✗	231	✗	276	infeas	321†	✓
7	✗	52	✓	97	✗	142	✓	187	✓	232	✗	277	infeas	322†	✓
8	✓	53	✗	98	✗	143	✓	188	✓	233	✓	278	✓	323†	✓
9	✓	54	✓	99	✓	144	✓	189	✓	234	✓	279	✓	324†	✓
10	✓	55	✓	100	✓	145	✗	190	✓	235	✓	280	✓	325†	✓
11	✓	56	✓	101	✓	146	✓	191	✓	236	✓	281	✓	326†	infeas
12	✓	57	infeas	102	✓	147	✓	192	✓	237	infeas	282	✓	327†	✓
13	✓	58	infeas	103	infeas	148	✓	193	✓	238	infeas	283	✓	328†	✓
14	✗	59	✓	104	✓	149	✓	194	✓	239	✓	284	✓	329†	✓
15	✓	60	✓	105	✓	150	✗	195	✓	240	infeas	285	✓	330†	✓
16	✓	61	✗	106	✓	151	✗	196	✓	241	✓	286	✓	331†	✗
17	✓	62	✓	107	✓	152	✗	197	✓	242	✓	287	✓	332†	✓
18	✓	63	✓	108	✓	153	✓	198	✓	243	✗	288	✓	333†	✓
19	✓	64	✓	109	✓	154	✓	199	✗	244	infeas	289	✓	334†	✓
20	✓	65	✓	110	✓	155	✓	200	✓	245	✗	290†	✓	335†	✓
21	✓	66	✓	111	✓	156	✗	201	✓	246	✓	291†	✓	336†	✓
22	✓	67	✓	112	✓	157	✗	202	✓	247	✓	292†	✓	337†	✓
23	✓	68	infeas	113	✓	158	✓	203	✓	248	✓	293†	✓	338†	✗
24	✓	69	✗	114	✓	159	✓	204	✓	249	✓	294†	✓	339†	✓
25	✓	70	✓	115	✓	160	✗	205	✓	250	✓	295†	✓	340†	✓
26	✓	71	✗	116	✓	161	✓	206	✓	251	infeas	296†	✓	341†	✓
27	✓	72	✗	117	✓	162	✗	207	✓	252	✓	297†	infeas	342†	✓
28	infeas	73	✗	118	✗	163	✓	208	✓	253	✗	298†	✓	343†	✓
29	✓	74	✓	119	✓	164	✓	209	✓	254	✓	299†	✗	344†	✓
30	✓	75	✓	120	✗	165	✓	210	✓	255	✓	300†	✓	345†	✗
31	✓	76	✓	121	✓	166	✓	211	✓	256	✓	301†	✓	346†	✗
32	✓	77	infeas	122	✓	167	✓	212	✓	257	✓	302†	✓	347†	✓
33	✓	78	✗	123	infeas	168	✗	213	✗	258	✓	303†	✗	348†	✓
34	✓	79	infeas	124	✓	169	✓	214	✓	259	✓	304†	✗	349†	✗
35	✓	80	✓	125	✓	170	✗	215	✓	260	✗	305†	✗	350†	✓
36	✓	81	✓	126	infeas	171	infeas	216	✗	261	infeas	306†	✓	351†	✓
37	✓	82	✓	127	✓	172	✗	217	✗	262	✗	307†	✓	352†	✓
38	✓	83	infeas	128	✓	173	✓	218	✓	263	✓	308†	✗	353†	✓
39	✓	84	✓	129	✓	174	✓	219	infeas	264	✓	309†	✓	354†	✓
40	✓	85	✓	130	✓	175	✓	220	✓	265	✓	310†	✗		
41	✓	86	✓	131	✓	176	✗	221	✗	266	infeas	311†	infeas		
42	✗	87	✗	132	✗	177	✗	222	✓	267	infeas	312†	infeas		
43	✓	88	✗	133	✓	178	✗	223	✓	268	✗	313†	✗		
44	✗	89	✓	134	✓	179	infeas	224	infeas	269	✓	314†	✗		
45	✓	90	✗	135	infeas	180	✓	225	✗	270	✓	315†	infeas		

Parameter Extraction. The model often confuses parameters with variables, misidentifies parameter shapes or misses some parameters. We use the following reflective prompt to correct these errors:

- *"Is the value of P known or not? Based on that, is it a parameter or a variable?"* (see Fig. 11)

Constraint Extraction. During constraint extraction, the model sometimes extracts trivial or vague constraints and may regenerate a previously generated constraint in different language. We use the following reflective prompts to correct these errors:

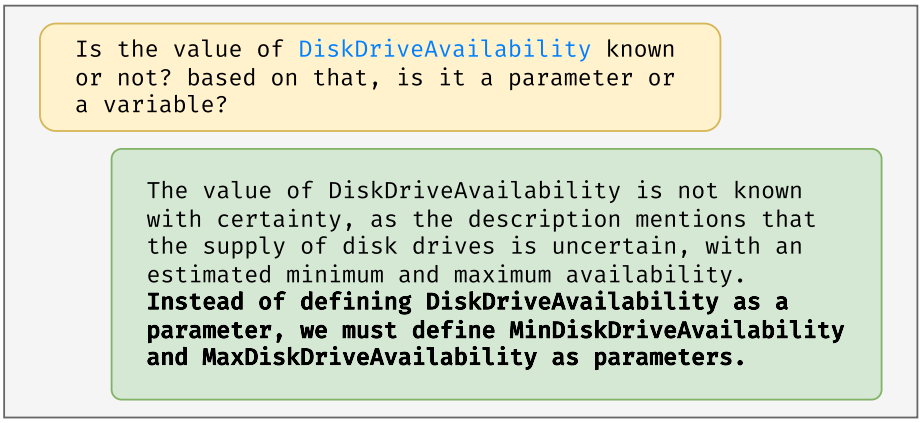


Figure 11 **OptiMUS-0.3 can fix parameter identification errors when prompted *"Is the value of P known or not?"***

> Does constraint C need to be explicitly modeled in the mathematical formulation? (see Fig. 15)
>
> Here's the list of all constraints: $C_1, C_2, \ldots, C_k$. Are any of them redundant?

Clause Modeling. Once the clauses are extracted, OptiMUS-0.3 models them, defining variables and auxiliary constraints if needed. Often, the model generates incorrect mathematical formulations for constraints. We use the following reflective prompts to detect and correct these errors:

> Are units the same for both sides of C? (see Fig. 3)
>
> Is C a correct constraint in the context of MILP solvers? (see Fig. 13)
>
> Does constraint C involve any variables, or is it only defined over parameters?

OptiMUS-0.3 uses similar reflective prompts to perform error checking for the optimization objective. For variables, we use the following prompt:

> Is the value of V known or not? Based on that, is it a parameter or a variable? (see Fig. 14)

Are units the same for both sides of this constraint?

$(p_a + x_a) \cdot d_a \cdot (1 + e_a \cdot \frac{x_a}{p_a}) \leq m_a, \forall a \in A$?

... Left-hand side (LHS):

- $(p_a + x_a)$ represents the new price for article a, which is in euros (€).

- d_a represents the sales forecast (demand) for article a for the next twelve months at the current price, which is in units of the article.

- $(1 + e_a \cdot \frac{x_a}{p_a})$ is a unitless factor ...

Therefore, the unit of the left-hand side is: **euros (€) × units of the article**

Right-hand side (RHS):

- m_a represents the maximum production volume for article a, which is in units of the article (e.g., number of shirts, pants, etc.).

The unit of the right-hand side is: **units of the article**

... this inconsistency suggests an error in the formulation of Constraint 5. To correct this, we should ... here is the corrected constraint:

$$d_a \cdot (1 + e_a \cdot \frac{x_a}{p_a}) \leq m_a, \forall a \in A$$

Figure 12 **OptiMUS-0.3 can fix its constraint modeling errors when prompted "*Are units the same for both sides of C?*"**

D. Additional details on Structure Detection

Table 11 shows the distribution of special problem structures in the NLP4LP dataset.

Table 11 **Distribution of problem structures in NLP4LP**

Structure	Number of instances
SOS	2
Indicator	6
Piecewise-linear	17

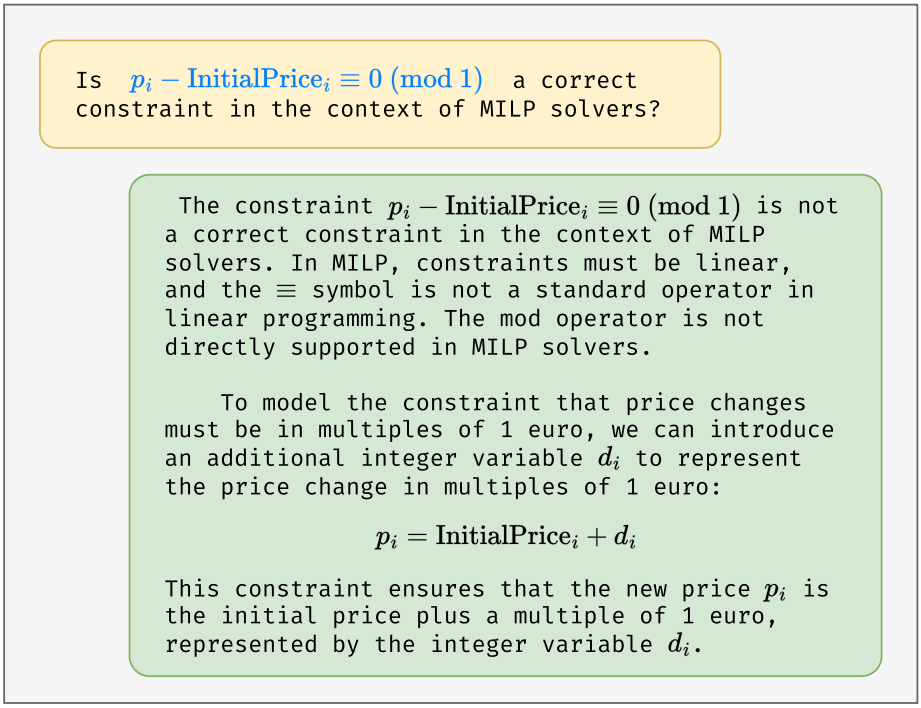


Figure 13 **Through introspection, OptiMUS-0.3 can fix its errors in constraint modeling.**

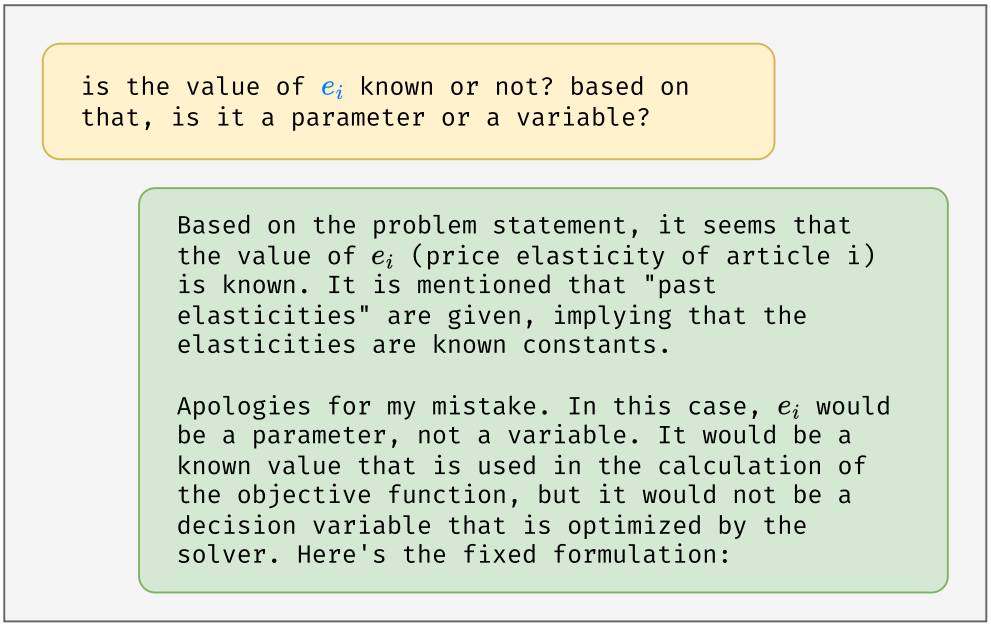


Figure 14 **OptiMUS-0.3 can fix its parameter and variable identification errors when prompted *"Is the value of P known or not?"***

Does constraint *Each system uses an integer number of disk drives.* need to be explicitly modeled in the mathematical formulation?

This statement seems to imply that we need to ensure that the number of disk drives used by each system is an integer. However, this is already implicit in the variable definition, as we are dealing with a discrete number of disk drives.

Therefore, I don't think this statement needs to be explicitly modeled as a constraint in the (MI)LP formulation.

Figure 15 **OptiMUS-0.3 can fix its parameter and variable identification errors when prompted *"Is the value of P known or not?"***

Example: Indicator Constraint

Definition

An indicator constraint over a binary variable z and a constraint C states that $z = 1 \Rightarrow C$.

Relevant description

A retail company wants to open new stores in a set of K potential locations. A store at location k must have a minimum staff of n_k employees (to ensure adequate customer service) and a minimum inventory level of l_k units (to avoid stockout). The maximum inventory level for store k is u_k, and there is a fixed cost associated with opening each store.

Relevant Constraints

If a store at location k is open, it must have at least n_k employees. Moreover, let the inventory level of store k be y_k.

Formulations

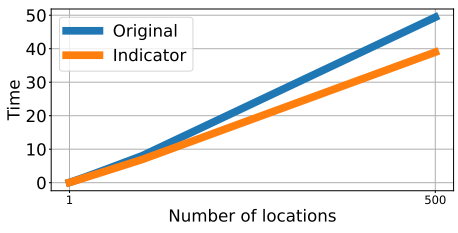
Original:

$$x_k \in \{0,1\},\ y_k \le u_k x_k,$$
$$z_k \ge l_k - M(1-x_k)$$

OptiMUS:

$$x_k \Rightarrow y_k \le u_k$$
$$x_k \Rightarrow z_k \ge l_k$$

Performance plot



Example: Special Ordered Set

Definition

Given a set of variables $\mathcal{X} = \{x_1, \ldots, x_n\}$,

- $\{x_1, \ldots, x_n\} \in \text{SOS}_1$ if at most one element of $\mathcal{X}$ can be nonzero
- $\{x_1, \ldots, x_n\} \in \text{SOS}_2$ if at most two elements of $\mathcal{X}$ can be nonzero

Relevant Description

An airline needs to assign N crew members to a set of K flights for a day. Each crew member can only be assigned to at most one flight at a time, and there must be a minimum rest period of 2 hours between consecutive flights to comply with aviation regulations and ensure crew well-being. Each crew member has a maximum number of flying hours per day, set at 8 hours, to comply with aviation regulations and prevent fatigue. There are minimum and maximum layover times between flights: minimum layover of 2 hours (to allow for crew rest and flight preparations) and maximum layover of 6 hours (to optimize crew utilization and reduce idle time). Assigning a crew member to a specific flight has some cost.

Relevant Constraints

Each crew member can work on only one flight. Let $x_{ij} \in \{0, 1\}$ denote whether crew i works on flight j and T_{ij} denote the working time of crew i on flight j.

Formulations

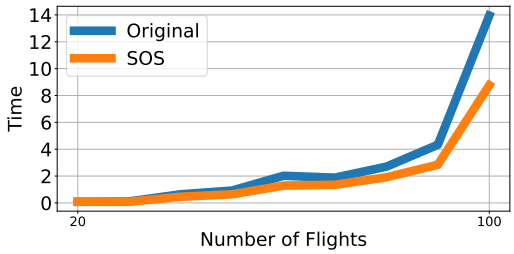
Original:

$$\sum_{j=1}^{K} x_{ij} \leq 1, \quad \forall i \in [N],$$
$$T_{ij} \leq 24 x_{ij}, \quad \forall i \in [N], j \in [K]$$

OptiMUS:

$$\{T_{i1}, \ldots, T_{iK}\} \in \text{SOS}_1$$

Performance plot



E. SCUC problem

Description. The Security Constrained Unit Commitment (SCUC) is a critical optimization problem in the operation of electrical power systems. Its goal is to schedule generation units in a cost-effective manner while ensuring reliable operation over a specified time period (typically 24 hours), considering the system's physical and operational constraints. This appendix presents a detailed description of the SCUC problem, including its objectives and constraints:
Objective. The objective of the SCUC problem is to minimize the total operational cost of the power system. This cost generally includes:

- Fuel Cost: The cost associated with consuming fuel to generate electricity.
- Start-up and Shut-down Cost: Cost incurred from starting up or shutting down generation units.
- Emission Cost: Cost related to the emissions produced by the generating units, if applicable.

Constraints. The SCUC must satisfy several constraints to ensure the safe and reliable operation of the power system:

- Power Balance Constraint: Ensures that the total power generation meets the total demand plus system losses at every interval. Mathematically, this is expressed as the sum of the outputs of all online generators being equal to the sum of the demand and the transmission losses.
- Minimum and Maximum Output Limits: Each generator has a minimum and maximum generation capacity when it is online. The SCUC ensures that the output of each unit stays within these bounds.
- Ramp-up and Ramp-down Limits: These limits specify the maximum rate at which a generator can increase or decrease its output. They are critical for handling load changes throughout the day.
- Unit Start-up and Shut-down Constraints: These constraints handle the logistics of turning units on or off. Start-up constraints may include minimum down times (the minimum time a unit must remain off before it can be restarted) and minimum up times (the minimum time a unit must remain on once started).
- Minimum Up and Down Time Constraints: Ensures that once a generator is turned on, it stays on for at least its minimum up time, and similarly, once turned off, it remains off for at least its minimum down time. These constraints are crucial for the mechanical integrity of the generation units.
- Reserve Requirements: The system operator must ensure that sufficient spinning and non-spinning reserves are available. These reserves are needed to handle sudden increases in demand or unexpected generator failures.

F. Web Application

As discussed in 1, the OptiMUS project aims to 1) help optimization experts develop and maintain their models with ease and 2) empower domain experts to make better decisions faster and reduce costs. To further these goals, we designed and built a user-centric web app with an intuitive interface that enables users to interact seamlessly with the system and leverage the advantages of large language models (LLMs).

We had the following key characteristics in mind when designing the web app:

- **User-centered design:** The design centers around the user's modeling flow rather than requiring users to adapt their workflow to the system.
- **Observability and intervention:** Given that LLM-based systems often are prone to errors, user supervision is critical. The interface is designed to facilitate this in every step.

The process includes several key steps:

1. **Parameter Extraction:** Users can input text in any format, and the model automatically extracts parameters. Users have the flexibility to edit and update parameters if necessary (Fig. 16).
2. **Clause Detection:** The model detects the objective and constraints given the provided information (Fig. 17).
3. **Clause Formulation:** Each clause is formulated by the model (Fig. 18).
4. **Code Generation:** The model generates code for each formulated clause (Fig. 19).
5. **Data File Structure Inference:** Based on the parameter information, a data file structure is inferred. Users can upload data in this format or use randomly generated data to continue (Fig. 20).
6. **Code Editing and Testing:** Users can edit the generated code, run it on the dataset, and debug it with assistance from the LLM (Figs. 21 and 22).

By allowing users to observe and provide input throughout each step, the web app brings significant speed and convenience to the modeling process while minimizing error risks.

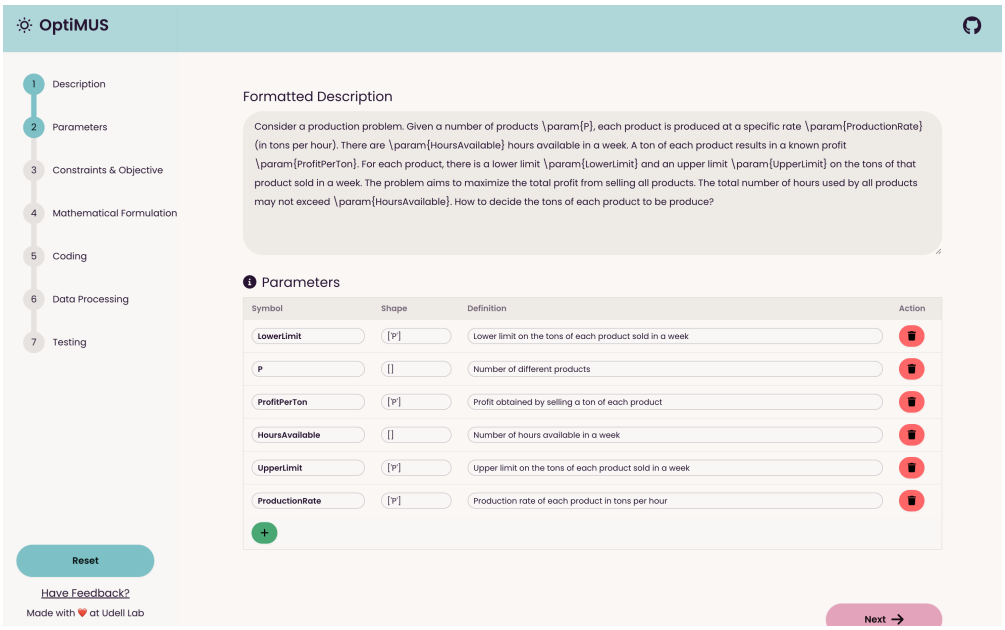


Figure 16 Parameter Identification: The parameters are automatically extracted, and users can add, remove, or modify them as needed.

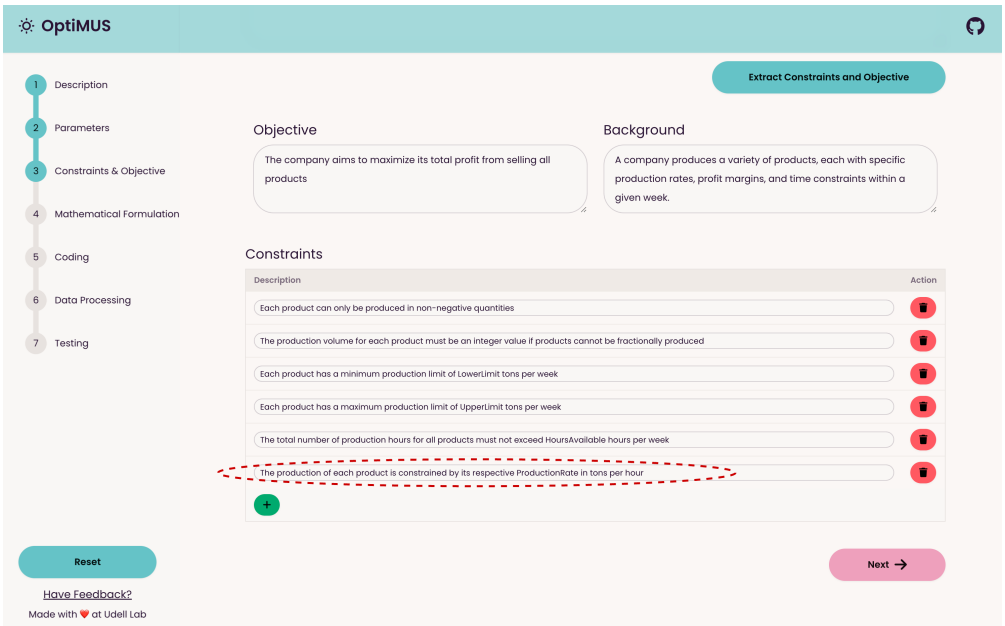


Figure 17 Clause Detection: In this example, one identified constraint is incorrectly marked as a constraint (The production limit is automatically enforced and does not need to be an explicit constraint). The interface allows the user to easily remove it.

G. Dataset Figures

Fig. 24 and Fig. 25 represent the number of instances in the dataset per industry sectors and operational areas. Resource allocation is the most common area and manufacturing is the most common sector. Figure 23 shows the distribution of



Figure 18 **Clause Formulation: An error is identified where one constraint is incorrect, allowing the user to correct it.**



Figure 19 **Code Generation for each clause based on the formulated constraints and objectives.**

instances by the number of associated labels with the problem. Most instances have around 4 labels, with some having as few as 2 and as many as 8

For plagiarism tests we use Papers Owl online tool PapersOwl (2024). See Fig. 26 for an example and Section G for a histogram of originality scores for all instances.

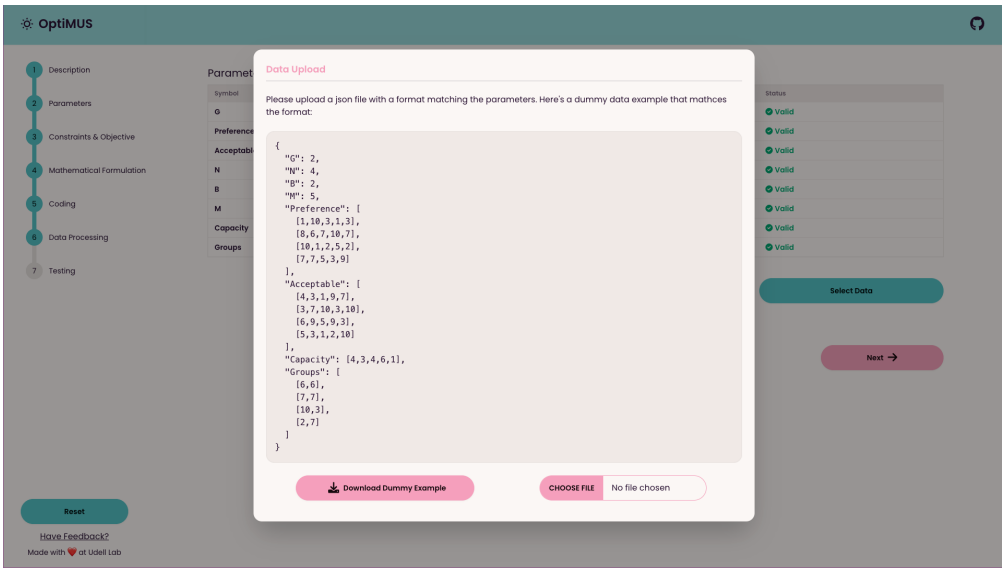


Figure 20 Data File Structure: Inferred from the extracted parameters, with options for data upload or random generation.

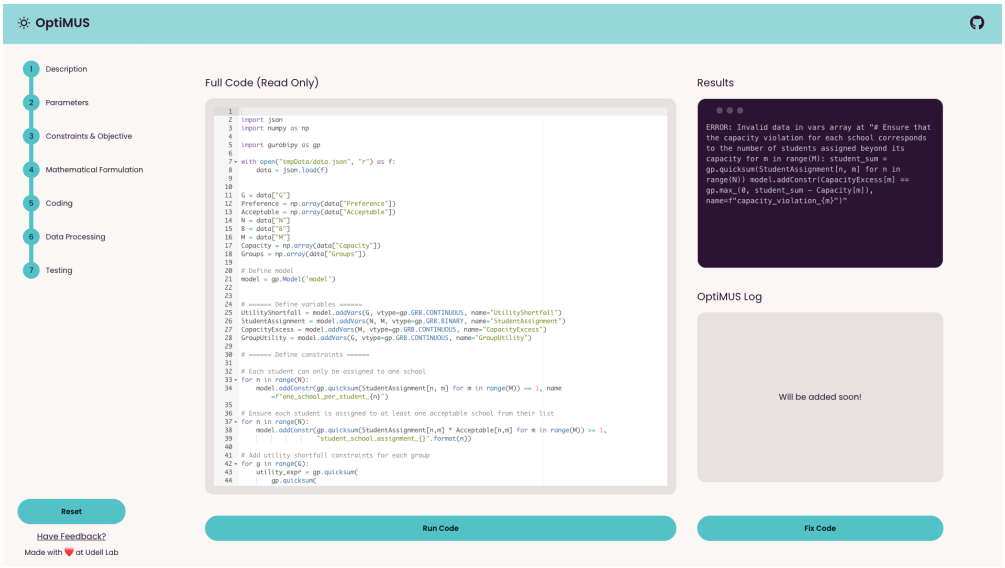


Figure 21 Testing Phase: The code is debugged and revised as errors are detected.

H. Inference Settings

For reproducibility, we report the exact inference settings used for all LLM calls in our experiments. No inference parameters were set explicitly in the OptiMUS code; all values listed below reflect the API defaults for each model at the time of our experiments.

† Temperature and top-p are fixed at 1.0 for o1 and cannot be adjusted via the API.

‡ Temperature and top-p are not supported parameters for o3; reasoning depth is controlled internally by the model.

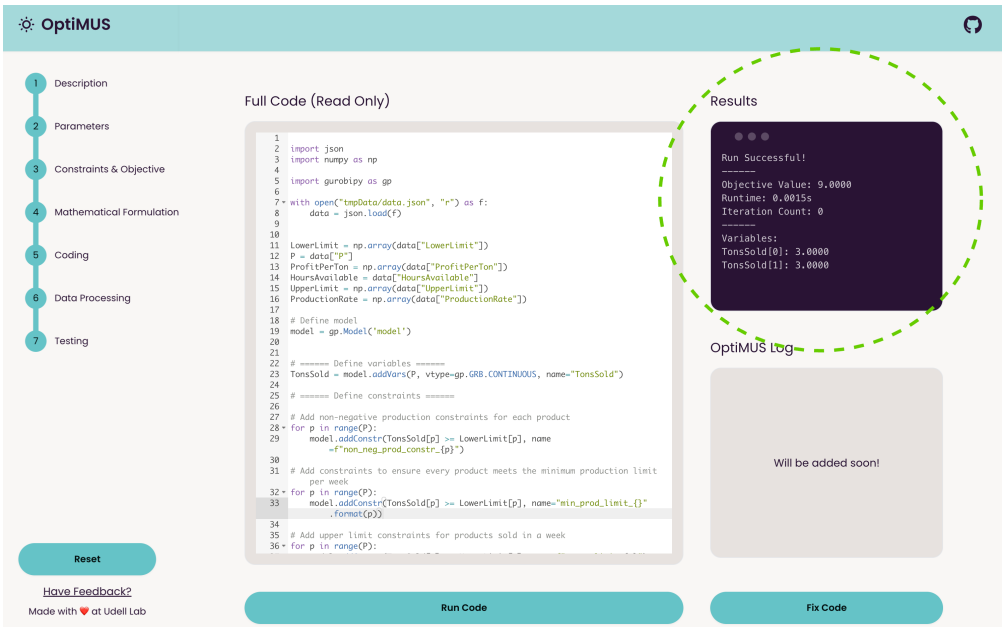


Figure 22 **Final Output: After debugging, the correct code is executed and the solution is displayed.**

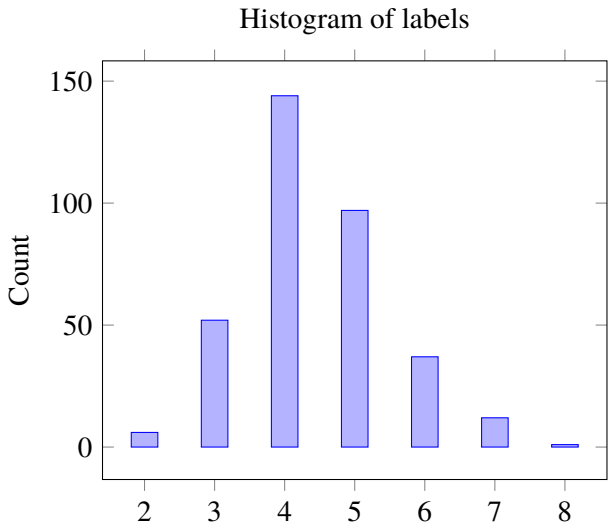


Figure 23 **Most problems have around 4 labels.**

Table 12 **Inference settings for all LLM calls in OptiMUS experiments.**

Use	Model (version)	Temperature	Top-p	Max Output Tokens
Main pipeline	GPT-4o (gpt-4o-2024-11-20)	1.0	1.0	16,384
Parameter extraction	o1	1.0†	1.0†	100,000
OptiMUS+o3 experiments	o3	N/A‡	N/A‡	100,000
Open-source experiments	LLaMA-3.1-70B-Instruct (llama3-70b-8192)	1.0	1.0	8,192

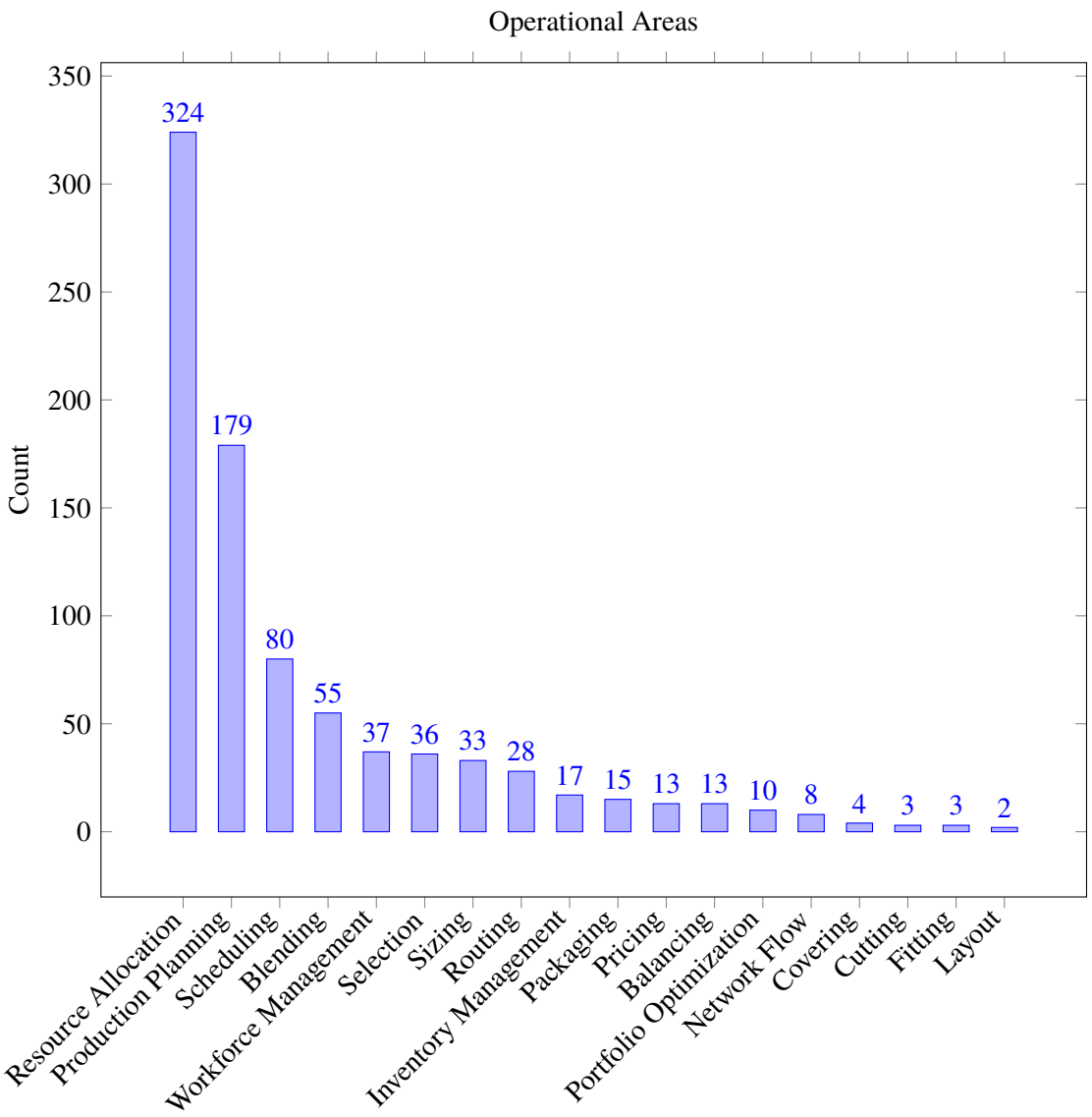


Figure 24 **NLP4LP covers common optimization problem types**

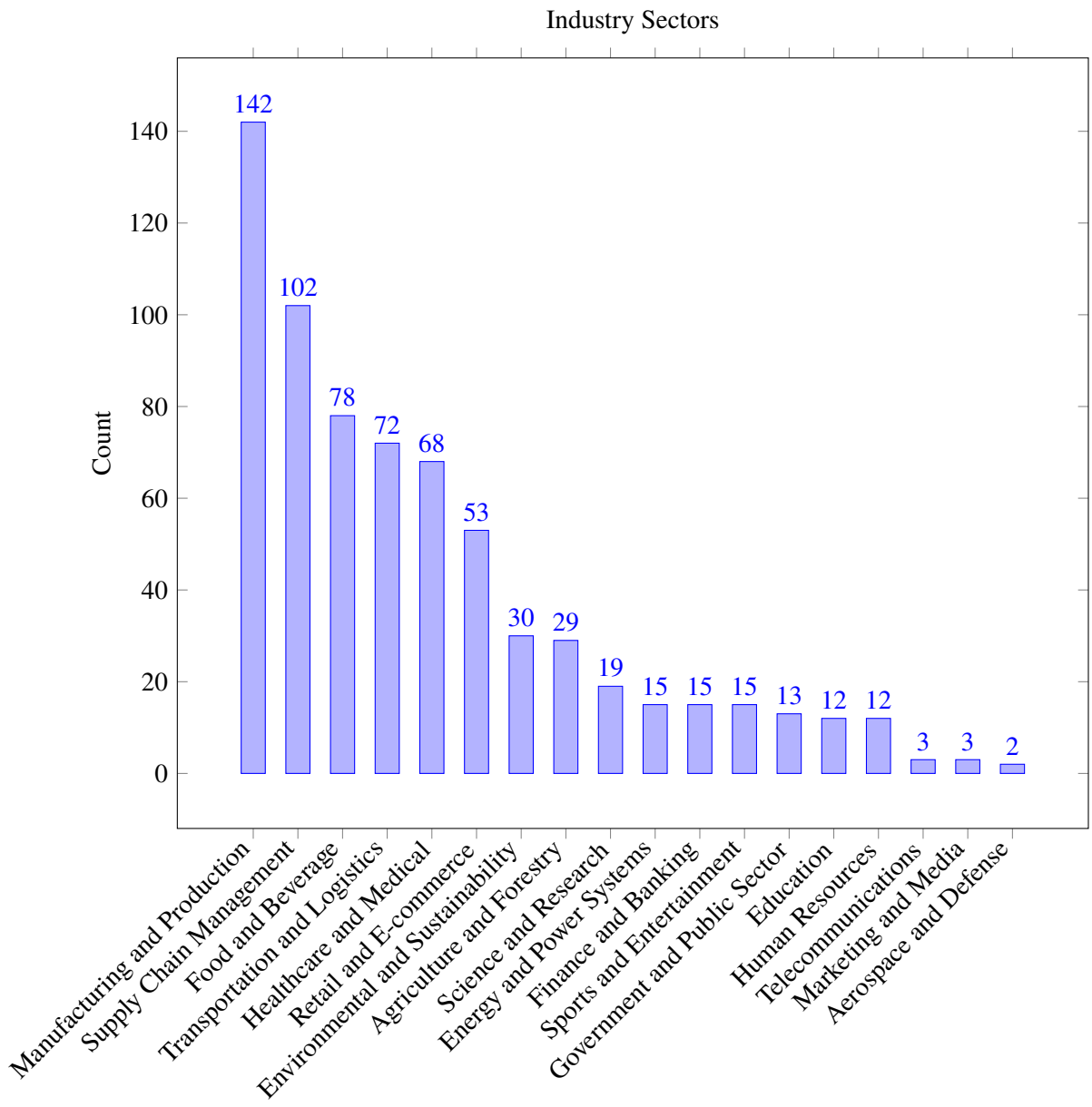


Figure 25 NLP4LP covers important optimization problem domains

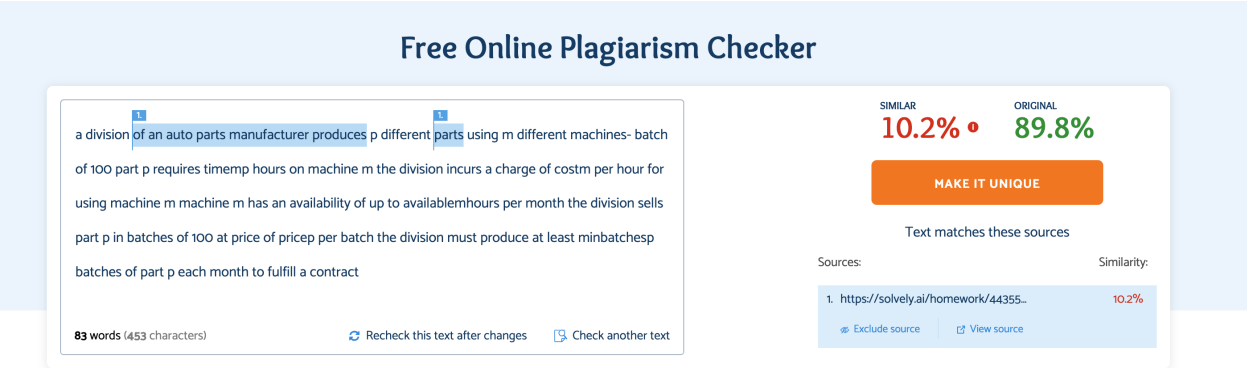


Figure 26 **We used plagiarism detection tools to ensure that instances are novel and publicly available on the internet. Most of the similarities detected are at word level (e.g. from a news article about manufacturing, or from a different optimization problem in the same domain).**

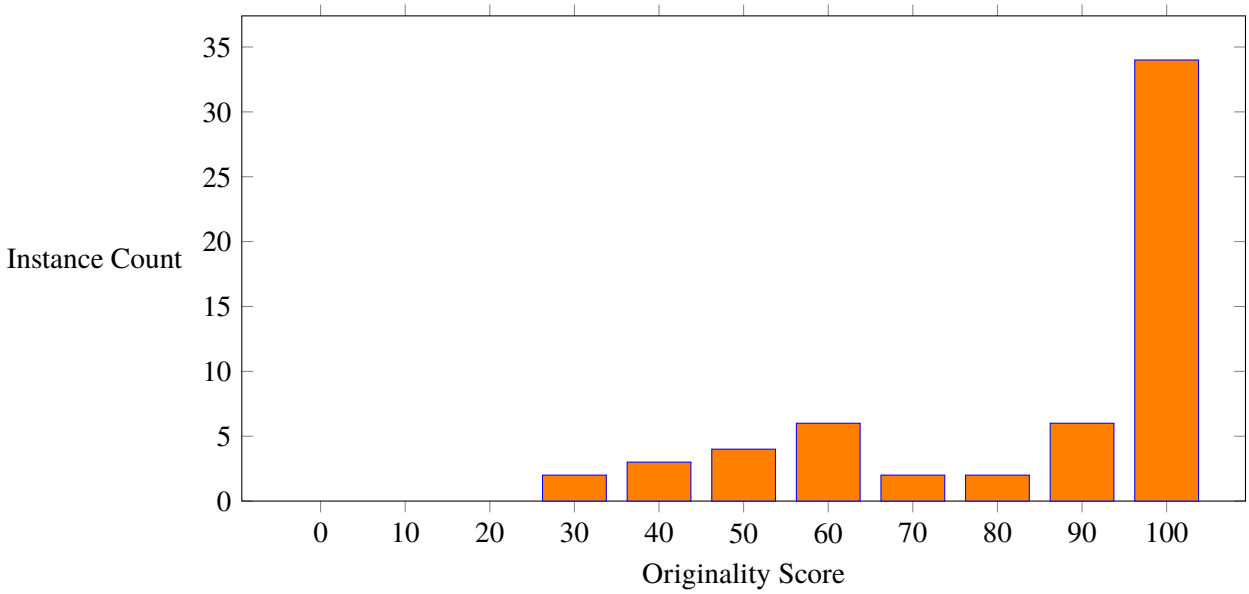


Figure 27 **Originality of the NLP4LP dataset. The instances in our dataset represent a really high originality score overall, minimizing the chances of them being using in internet-scraped training corpus used for training LLMs.**